



Unidade Curricular de Base de Dados 2025/2026

Relatório de Projeto : EvenSync

**André Marques - A111634 , Bruno Freitas - A110016 ,
João Ribeiro - A111041 , Tiago Santos - A110313 , Nelson
Antunes - A110360**

Data de Entrega : 15 de Maio de 2026

Abstract

António Carlos Silva Abelha demonstrou, ao longo da sua carreira académica, uma grande paixão por partilhar conhecimento e organizar eventos. Contudo, a falta de uma estrutura centralizada provocava a perda de detalhes importantes e dificultava a gestão de inscrições e de pagamentos. Perante esta situação, considerou que a solução para o problema passaria pela criação de um Sistema de Gestão de Bases de Dados (SGBD) que permitisse armazenar e organizar de forma estruturada toda a informação da sua plataforma *EvenSync*, tendo encomendado o seu desenvolvimento a um grupo de alunos da Licenciatura em Engenharia Informática da Universidade do Minho.

Este documento descreve o processo de desenvolvimento realizado pela equipa de alunos. Numa fase inicial, o cliente expôs a necessidade de criação de uma base de dados e definiu o contexto em que esta seria desenvolvida. Seguiu-se a definição do plano de execução do projeto e do método de levantamento de requisitos. Os requisitos foram recolhidos e consolidados através de uma abordagem híbrida que combinou entrevistas detalhadas com o cliente, análise documental, *benchmarking* de mercado e sessões de *brainstorming* da equipa, sendo posteriormente organizados e validados de forma estruturada.

Com base nos requisitos identificados, procedeu-se à criação e documentação de um diagrama Entidade-Relacionamento (ER), que foi posteriormente transformado num modelo lógico relacional. Este modelo foi normalizado até à Terceira Forma Normal (3FN), de modo a garantir a integridade dos dados e a redução de redundância. A validação do modelo lógico foi realizada através da definição e análise de interrogações recorrendo à álgebra relacional.

O modelo lógico validado foi convertido para um modelo físico no Sistema de Gestão de Bases de Dados MySQL. Nesta fase foram criados utilizadores da base de dados com diferentes níveis de privilégio, procedeu-se ao povoamento da base de dados e à implementação das interrogações em SQL. Adicionalmente, foram definidas vistas de utilização, índices, bem como alguns procedimentos, funções e gatilhos essenciais, com o objetivo de garantir o cumprimento das regras de negócio, melhorar o desempenho das consultas e assegurar a consistência dos dados armazenados.

Área de Aplicação: Desenho e Arquitetura de Sistemas de Bases de Dados

Palavras-Chave: Bases de Dados, Diagramas Entidade-Relacionamento, Modelo Relacional, Normalização de Bases de Dados Relacionais, Álgebra Relacional, SQL, MySQL

Índice

1	Definição do Sistema	1
1.1	Contexto de aplicação	1
1.2	Motivação e Objetivos do Trabalho	1
1.3	Análise de Viabilidade do Processo	2
1.4	Recursos e Equipa de Trabalho	3
1.5	Plano de Execução do Projeto	3
2	Levantamento e Análise de Requisitos	5
2.1	Método de Levantamento e de Análise de Requisitos Adotado . .	5
2.2	Organização dos Requisitos Levantados	6
2.2.1	Critérios de Classificação	6
2.2.2	Organização dos Requisitos Levantados	6
2.2.3	Resumo dos Requisitos Identificados	10
2.2.4	Ligação à Fase Seguinte	10
2.3	Análise e Validação Geral dos Requisitos	10
3	Modelação Conceptual	12
3.1	Apresentação da Abordagem de Modelação Realizada	12
3.2	Identificação e Caracterização das Entidades	13
3.2.1	Entidade: Evento	14
3.2.2	Entidade: Organizador	15
3.2.3	Entidade: Sessão	15
3.2.4	Entidade: Orador	16
3.2.5	Entidade: Participante	16
3.2.6	Entidade: Inscrição	17
3.2.7	Entidade: Pagamento	17
3.3	Identificação e Caracterização dos Relacionamentos	19
3.3.1	Visão Global dos Relacionamentos	19
3.3.2	Caracterização Detalhada dos Relacionamentos	20
3.4	Identificação e Caracterização dos Atributos das Entidades e dos Relacionamentos	24
3.4.1	Entidade: Participante	24
3.4.2	Entidade: Inscrição	25
3.4.3	Entidade: Pagamento	26
3.4.4	Entidade: Evento	27
3.4.5	Entidade: Organizador	28
3.4.6	Entidade: Sessão	29
3.4.7	Entidade: Orador	30
3.5	Apresentação e Explicação do Diagrama ER Produzido	31

4	Modelação Lógica	32
4.1	Construção e Validação do Modelo de Dados Lógico	32
4.1.1	Abordagem de Construção do Modelo Lógico	32
4.1.2	Estrutura Geral do Modelo Lógico	32
4.1.3	Validação do Modelo de Dados Lógico	33
4.2	Apresentação e Explicação do Modelo Lógico Produzido	34
4.2.1	Relação: Participante	36
4.2.2	Relação: Evento	37
4.2.3	Relação: Organizador	37
4.2.4	Relação: Sessao	38
4.2.5	Relação: Orador	38
4.2.6	Relação: Inscricao	39
4.2.7	Relação: Pagamento	39
4.2.8	Relação: Gere	40
4.2.9	Relação: Apresentada	40
4.3	Normalização de Dados	40
4.3.1	Dependências Funcionais e Chaves Identificadas	40
4.3.2	Verificação das Formas Normais	41
4.3.3	Anomalias Evitadas	42
4.3.4	Conclusão	43
4.4	Validação do Modelo com Interrogações do Utilizador	43
5	Implementação Física	45
5.1	Apresentação e Explicação da Base de Dados Implementada	45
5.2	Criação de Utilizadores da Base de Dados	46
5.3	Povoamento da Base de Dados	47
5.4	Cálculo do Espaço da Base de Dados (Inicial e Taxa de Crescimento Anual)	47
5.5	Definição e Caracterização de Vistas de Utilização em SQL	53
5.6	Tradução das Interrogações do Utilizador para SQL	54
5.7	Indexação do Sistema de Dados	56
5.8	Implementação de Procedimentos, Funções e Gatilhos	57
5.8.1	Gatilho: <code>trg_verificar_sala_disponivel</code> (RC6)	57
5.8.2	Procedimento: <code>sp_confirmar_pagamento</code>	58
5.8.3	Função: <code>fn_total_gasto_participante</code>	58
6	Conclusão e Trabalho Futuro	59
7	Bibliografia	60
A	Script DDL — Criação do Esquema Físico	62
B	Script de Povoamento	67
C	Vistas, Gatilho, Procedimento, Função e Interrogações SQL	70
D	Diagrama Entidade-Relacionamento (tamanho completo)	74

List of Figures

1	Diagrama de Gantt do Projeto EvenSync (Março – Maio 2026)	4
2	Relacionamento <i>realiza</i> entre Participante e Inscrição	20
3	Relacionamento <i>pertence</i> entre Inscrição e Evento	20
4	Relacionamento <i>precisa</i> entre Inscrição e Pagamento	21
5	Relacionamento <i>gere</i> entre Organizador e Evento	22
6	Relacionamento <i>tem</i> entre Evento e Sessão	22
7	Relacionamento <i>apresentada</i> entre Sessão e Orador	23
8	Diagrama Entidade-Relacionamento do sistema EvenSync	31
9	Diagrama do Modelo Lógico do sistema EvenSync (gerado com MySQL Workbench)	35
10	Diagrama ER completo do sistema EvenSync (notação de Chen)	74
11	Modelo lógico do sistema EvenSync (gerado com MySQL Workbench)	75

List of Tables

1	Requisitos de Descrição do EvenSync	7
2	Requisitos de Manipulação do EvenSync	8
3	Requisitos de Controlo do EvenSync	9
4	Entidades do Sistema EvenSync	13
5	Síntese dos Relacionamentos do Sistema EvenSync	19
6	Atributos da entidade Participante	24
7	Atributos da entidade Inscrição	25
8	Atributos da entidade Pagamento	26
9	Atributos da entidade Evento	27
10	Atributos da entidade Organizador	28
11	Atributos da entidade Sessão	29
12	Atributos da entidade Orador	30
13	Dicionário de dados da relação Participante	36
14	Dicionário de dados da relação Evento	37
15	Dicionário de dados da relação Organizador	37
16	Dicionário de dados da relação Sessao	38
17	Dicionário de dados da relação Orador	38
18	Dicionário de dados da relação Inscricao	39
19	Dicionário de dados da relação Pagamento	39
20	Dicionário de dados da relação Gere	40
21	Dicionário de dados da relação Apresentada	40
22	Tabelas do modelo físico do EvenSync	45
23	Utilizadores da base de dados EvenSync e respetivos privilégios .	46
24	Regras de ocupação dos tipos de dados no MySQL 8	48
25	Espaço por linha na tabela Participante	48
26	Espaço por linha na tabela Evento	49
27	Espaço por linha na tabela Organizador	49
28	Espaço por linha na tabela Sessao	50
29	Espaço por linha na tabela Orador	50
30	Espaço por linha na tabela Inscricao	51
31	Espaço por linha na tabela Pagamento	51
32	Espaço por linha nas tabelas associativas	51
33	Estimativa global do espaço inicial da base de dados EvenSync .	52
34	Taxa de crescimento anual estimada da base de dados EvenSync	53
35	Índices adicionais definidos no EvenSync	56
36	Elementos de lógica implementados no EvenSync	59

1 Definição do Sistema

1.1 Contexto de aplicação

António Carlos Silva Abelha, professor universitário durante mais de três décadas, sempre nutriu uma paixão pela partilha de conhecimento e pelos eventos académicos que ao longo da sua carreira organizou e frequentou.

Após a sua reforma, ao continuar ligado ao mundo dos eventos, deparou-se repetidamente com um problema que conhecia bem: a organização de eventos era, na sua grande maioria, fragmentada e ineficiente, assente em processos manuais e ferramentas dispersas que dificultavam a gestão dos participantes, horários e recursos.

Determinado a encontrar uma solução, António Carlos Silva Abelha decidiu colocar a sua experiência ao serviço de um novo projeto e criar uma plataforma própria de gestão de eventos — assim nasceu a EvenSync.

Ciente de que um projeto desta natureza exigia uma base sólida de dados, António Carlos Silva Abelha estabeleceu uma parceria com a equipa de desenvolvimento da Universidade do Minho, que se propôs a desenvolver um Sistema de Gestão de Base de Dados (SGBD) relacional capaz de suportar toda a informação associada ao EvenSync. O objetivo seria criar uma estrutura que garantisse a organização, integridade e acessibilidade dos dados, permitindo ao mesmo tempo a gestão eficaz de eventos de diferentes tipologias e a interação entre os vários intervenientes no processo.

1.2 Motivação e Objetivos do Trabalho

Atualmente, a maioria das organizações que promove eventos recorre a um conjunto fragmentado de ferramentas para gerir as diferentes componentes do processo: folhas de cálculo para controlo de inscrições, plataformas de email para comunicação com participantes, sistemas de pagamento independentes e documentos isolados para a gestão de salas e horários. Foi precisamente este cenário que António Carlos Silva Abelha, professor universitário durante mais de três décadas, encontrou ao continuar ligado ao mundo dos eventos após a sua reforma. Habitado a organizar e participar em conferências e seminários académicos, deparou-se repetidamente com a mesma realidade: a informação encontrava-se dispersa, sem qualquer ligação entre si, tornando-se cada vez mais difícil de manter — um participante inscrito não estava automaticamente associado à sua fatura, nem ao certificado que lhe devia ser emitido.

A falta de uma estrutura centralizada provocava não apenas a perda de contexto e detalhes importantes, mas também a impossibilidade de realizar análises eficazes sobre a atividade da organização. Dados sobre taxas de ocupação de salas, participação por tipo de evento ou receitas geradas ficavam inacessíveis ou exigiam um esforço manual considerável para serem compilados. Determinado

a resolver este problema, António Carlos Silva Abelha decidiu tomar a iniciativa e criar a sua própria solução.

Perante esta situação, e em parceria com a equipa de desenvolvimento da Universidade do Minho, considerou-se que a solução passaria por um Sistema de Gestão de Base de Dados que permitisse catalogar, gerir e consultar toda a informação associada aos eventos de forma estruturada e duradoura. Inspirados por esta necessidade, definiram-se os seguintes objetivos para o EvenSync:

- Estruturar a informação relacionada com eventos, participantes, oradores, salas, inscrições, pagamentos e certificados, suportando múltiplas tipologias de eventos em simultâneo;
- Facilitar o armazenamento e recuperação de dados, garantindo que a informação se mantém organizada, íntegra e facilmente acessível;
- Apoiar a gestão operacional dos eventos, fornecendo mecanismos de consulta, vistas e relatórios que respondam às necessidades dos utilizadores do sistema;
- Assegurar a integridade referencial dos dados através da definição rigorosa de restrições, procedimentos e gatilhos;
- Garantir o crescimento contínuo do sistema sem comprometer o seu desempenho, através de uma indexação adequada e de uma estimativa realista do espaço em base de dados.

1.3 Análise de Viabilidade do Processo

Atualmente, muitas organizações enfrentam dificuldades em manter um registo coerente e sustentável das suas atividades de eventos, que acabam dispersas entre ferramentas genéricas, comunicações por email e documentos físicos sem qualquer estrutura comum. Surge, assim, a necessidade de um sistema que centralize toda esta informação de forma duradoura e consultável.

Do ponto de vista **técnico**, o projeto é totalmente viável. As tecnologias selecionadas — nomeadamente o MySQL como SGBD relacional — são amplamente utilizadas, estáveis, bem documentadas e suportam facilmente o volume de dados previsto para uma aplicação desta natureza. A equipa possui os conhecimentos necessários para a sua utilização, adquiridos no âmbito da unidade curricular de Bases de Dados e de outras disciplinas do plano de estudos.

Do ponto de vista **económico**, o investimento necessário é praticamente nulo, uma vez que todas as ferramentas e tecnologias utilizadas são gratuitas ou de código aberto, e o projeto é desenvolvido em ambiente académico sem custos de infraestrutura adicional.

Do ponto de vista **temporal**, o projeto foi planeado para ser desenvolvido entre a data de lançamento do enunciado e o dia 10 de maio de 2026, prazo de entrega definido pelo docente. Este período considera-se suficiente para

completar todas as fases previstas, desde que o plano de execução definido na secção 1.5 seja respeitado.

Deste modo, considera-se que o EvenSync é um projeto viável e sustentável, tanto no âmbito técnico como académico.

1.4 Recursos e Equipa de Trabalho

No âmbito deste projeto, os recursos humanos estão divididos em duas categorias: **peçoal interno** e **peçoal externo**. O *peçoal interno* corresponde ao cliente — ou seja, a pessoa que identificou o problema, idealizou a solução e encomendou o seu desenvolvimento. O *peçoal externo* corresponde à equipa técnica que foi recrutada para concretizar o projeto.

O “peçoal interno” é representado pelo seguinte indivíduo:

- **António Carlos Silva Abelha:** Licenciado em Engenharia de Sistemas e Informática, Mestre em Informática de Gestão e Doutoramento em Informática pela Universidade do Minho. É o idealizador do projeto EvenSync. Após ter participado na organização de um seminário académico, confrontou-se com as dificuldades práticas da gestão manual de inscrições, presenças e certificados, o que o levou a conceber a ideia de uma plataforma centralizada de gestão de eventos. Ciente de que não conseguiria concretizar sozinho o projeto, procurou o apoio de uma equipa com competências técnicas em Bases de Dados.

O “peçoal externo” é constituído pela equipa de cinco alunos de Ciências da Computação recrutada pelo próprio António Carlos Silva Abelha, responsável pelo desenvolvimento do sistema, pela sua implementação e testagem. Apresentam conhecimentos sólidos em modelação de bases de dados, programação em SQL, design de sistemas de informação e metodologias de desenvolvimento. Os nomes dos integrantes desta equipa são André Marques, Bruno Freitas, João Ribeiro, Tiago Santos e Nelson Antunes, todos estudantes da Licenciatura em Ciências da Computação da Universidade do Minho. A equipa é responsável por todas as fases do projeto, desde a análise de requisitos até à implementação física da base de dados, garantindo que o sistema desenvolvido atende às necessidades do cliente e cumpre os objetivos definidos.

1.5 Plano de Execução do Projeto

A equipa reuniu-se com António Carlos Silva Abelha para planear o desenvolvimento do SGBD e definir a calendarização das tarefas. O plano foi estruturado em 6 fases com duração total de 9 semanas (março a maio de 2026), tendo em conta o prazo de entrega estipulado e os períodos de maior carga académica da equipa. A cada tarefa foi atribuído um responsável, garantindo uma distribuição equilibrada do trabalho. O diagrama de Gantt da Figura 1 permite visualizar a duração de cada fase, os responsáveis e as dependências sequenciais entre tarefas.

Id	Tarefa	Atribuído a:	Fase	S.1	S.2	S.3	S.4	S.5	S.6	S.7	S.8	S.9
1	Definição do Sistema		Fase									
1.1	Contextualização	André	1									
1.2	Motivação e Objetivos	Bruno	1									
1.3	Análise de Viabilidade	João	1									
1.4	Recursos e Equipa	Nelson	1									
1.5	Plano de Execução	Tiago	1									
2	Levantamento de Requisitos		Fase									
2.1	Método de Levantamento	André	2									
2.2	Organização de Requisitos	Bruno, João	2									
2.3	Validação de Requisitos	Nelson, Tiago	2									
3	Modelação Conceptual		Fase									
3.1	Abordagem de Modelação	André	3									
3.2	Identificação de Entidades	João	3									
3.3	Identificação Relacionamentos	Nelson	3									
3.4	Atributos	Tiago	3									
3.5	Diagrama ER Final	Bruno	3									
4	Modelação Lógica		Fase									
4.1	Construção Modelo Lógico	André	4									
4.2	Documentação do Modelo	Bruno	4									
4.3	Normalização	João	4									
4.4	Validação UCQs	Nelson, Tiago	4									
5	Implementação Física		Fase									
5.1	Scripts DDL	Tiago	5									
5.2	Utilizadores e Privilégios	Nelson	5									
5.3	Povoamento da BD	André	5									
5.4	Cálculo de Espaço	Bruno	5									
5.5	Vistas SQL	João	5									
5.6	UCQs em SQL	André	5									
5.7	Indexação	Nelson	5									
5.8	Procedures/Triggers	Tiago	5									
6	Revisão e Entrega Final		Fase									
6.1	Testes de Integridade	Todos	6									
6.2	Revisão Cruzada	Todos	6									
6.3	Redação Final	Bruno	6									
6.4	Submissão	Todos	6									

Figure 1: Diagrama de Gantt do Projeto EvenSync (Março – Maio 2026)

2 Levantamento e Análise de Requisitos

2.1 Método de Levantamento e de Análise de Requisitos Adotado

A definição de uma base de dados adequada ao contexto do EvenSync exigiu, numa primeira etapa, uma compreensão aprofundada das necessidades reais do cliente. Para tal, a equipa adotou uma abordagem híbrida e colaborativa, combinando **entrevistas estruturadas** com sessões de *brainstorming*. As entrevistas com António Carlos Silva Abelha, idealizador do projeto, permitiram capturar requisitos explícitos, preferências e restrições operacionais que dificilmente emergiriam em questionários escritos.

Estas entrevistas decorreram ao longo da terceira semana do plano de execução. Cada membro da equipa ficou responsável por conduzir a análise relativa a um subconjunto de funcionalidades do sistema. Posteriormente, a equipa reuniu-se em sessões de *brainstorming* para cruzar ideias, detetar requisitos implícitos (que o cliente não expressou diretamente, mas que são essenciais para a consistência do sistema) e resolver possíveis conflitos lógicos.

Para além destas abordagens interativas, o processo foi complementado por uma rigorosa **análise documental** do caso de estudo fundacional e pela técnica de *benchmarking*. A equipa analisou plataformas reais de mercado (como a *Eventbrite* e o *Meetup*) para aferir as melhores práticas da indústria, garantindo que não fossem negligenciados dados essenciais de registo ou características típicas da gestão profissional de eventos.

A combinação das entrevistas e *brainstorming* com a pesquisa documental e de mercado revelou-se altamente eficaz. O contacto com o cliente esclareceu ambiguidades funcionais, enquanto as sessões de debate interno e a análise de sistemas similares garantiram que a arquitetura planeada seria técnica e logicamente robusta. Após esta fase iterativa, os requisitos recolhidos foram validados e organizados de forma estruturada, conforme apresentado na subsecção seguinte.

2.2 Organização dos Requisitos Levantados

2.2.1 Critérios de Classificação

Concluída a fase de recolha, os requisitos foram sistematizados segundo três categorias distintas, seguindo a classificação clássica da engenharia de bases de dados: **Descrição** (toda a informação que o sistema deverá armazenar de forma persistente), **Manipulação** (as operações e consultas que o sistema deverá disponibilizar aos seus utilizadores) e **Controlo** (as restrições e regras de negócio que o sistema deverá impor de forma automática). A cada requisito foi atribuído um identificador único — RD para Descrição (Dados), RM para Manipulação e RC para Controlo — de modo a garantir a rastreabilidade do mesmo nas fases subsequentes do projeto.

2.2.2 Organização dos Requisitos Levantados

Após a aplicação do método de levantamento descrito na Secção 2.1, foi obtido um conjunto de requisitos para o sistema EvenSync. Posteriormente, os requisitos foram divididos em três tabelas distintas, de acordo com a sua categoria: Requisitos de Descrição (RD), Requisitos de Manipulação (RM) e Requisitos de Controlo (RC), para facilitar a consulta, validação e posterior utilização nas fases de modelação conceptual, lógica e implementação física.

ID	Requisitos de Descrição (RD)	Analista
RD1	O sistema deve permitir registar eventos com um identificador único, nome, tipo (conferências, workshops, eventos corporativos, seminários), número de edição, datas de início e fim, localização, capacidade máxima e descrição.	João Ribeiro
RD2	O sistema deve permitir registar organizadores com um identificador único, nome, email e telefone.	João Ribeiro
RD3	O sistema deve permitir associar um ou mais organizadores a cada evento, sendo que um organizador pode gerir múltiplos eventos.	João Ribeiro
RD4	O sistema deve permitir registar sessões associadas a um evento, com um identificador único, título, sala, data, hora de início, hora de fim e descrição.	João Ribeiro
RD5	O sistema deve permitir registar oradores com um identificador único, nome, email e especialidade.	Tiago Santos
RD6	O sistema deve permitir associar um ou mais oradores a cada sessão, sendo que um orador pode apresentar múltiplas sessões.	Bruno Freitas
RD7	O sistema deve permitir registar participantes com um identificador único, nome, email, telefone e organização ou instituição.	Bruno Freitas
RD8	O sistema deve permitir registar inscrições de participantes em eventos, com um identificador único, data de inscrição e estado (confirmada, pendente ou cancelada).	João Ribeiro
RD9	O sistema deve permitir registar pagamentos associados a inscrições, com um identificador único, valor, data de pagamento, método de pagamento e estado.	André Marques

Table 1: Requisitos de Descrição do EvenSync

ID	Requisitos de Manipulação (RM)	Analista
RM1	Deve ser possível consultar todos os eventos registados no sistema, com possibilidade de filtrar por tipo, data ou localização.	André Marques
RM2	Deve ser possível consultar o programa completo de um evento, listando todas as sessões com os respetivos horários, salas e oradores associados.	André Marques
RM3	Deve ser possível consultar todos os inscritos num determinado evento, bem como o estado da sua inscrição.	André Marques
RM4	Deve ser possível consultar o histórico de inscrições de um participante, incluindo os eventos em que participou e os respetivos estados.	Nelson Antunes
RM5	Deve ser possível consultar o total de receitas geradas por um evento, com base nos pagamentos com estado pago.	Nelson Antunes
RM6	Deve ser possível obter a lista de eventos com maior taxa de ocupação, calculada com base no rácio entre o número de inscrições confirmadas e a capacidade máxima do evento.	Nelson Antunes
RM7	Deve ser possível consultar os oradores que apresentam sessões de um determinado evento.	Nelson Antunes
RM8	Deve ser possível consultar todas as sessões agendadas numa determinada sala ou num intervalo de datas.	Tiago Santos
RM9	Deve ser possível consultar os oradores atribuídos a uma determinada sessão.	Bruno Freitas
RM10	Deve ser possível consultar os pagamentos com estado pendente ou rejeitado, identificando o participante e o evento associados.	André Marques

Table 2: Requisitos de Manipulação do EvenSync

ID	Requisitos de Controlo (RC)	Analista
RC1	O sistema deve garantir que um participante não pode ter mais do que uma inscrição ativa no mesmo evento.	Tiago Santos
RC2	O sistema deve garantir que cada inscrição tem, no máximo, um pagamento associado.	Tiago Santos
RC3	O sistema deve garantir que o email de cada participante, orador e organizador é único no sistema.	João Ribeiro
RC4	Apenas os organizadores podem criar, editar ou eliminar eventos.	João Ribeiro
RC5	Apenas os organizadores podem gerir os dados de participantes e emitir relatórios financeiros.	João Ribeiro
RC6	O sistema deve impedir o registo de sessões com sobreposição de horário na mesma sala e no mesmo evento.	João Ribeiro

Table 3: Requisitos de Controlo do EvenSync

2.2.3 Resumo dos Requisitos Identificados

O processo de levantamento resultou na identificação de um total de 25 requisitos, distribuídos da seguinte forma:

- Requisitos de Descrição (RD): 9 requisitos
- Requisitos de Manipulação (RM): 10 requisitos
- Requisitos de Controlo (RC): 6 requisitos

Os requisitos abrangem as 7 entidades principais do sistema identificadas para a plataforma: Evento, Organizador, Sessão, Orador, Participante, Inscrição e Pagamento. A análise e validação geral dos requisitos é apresentada na Secção 2.3.

2.2.4 Ligação à Fase Seguinte

Os requisitos levantados nesta fase constituem o alicerce direto da modelação conceptual apresentada na Secção 3. Cada entidade identificada no Diagrama Entidade-Relacionamento decorre diretamente de um ou mais requisitos de descrição (RD1–RD9), e cada restrição de cardinalidade ou regra de integridade definida no modelo corresponde a um requisito de controlo (RC1–RC6). Esta rastreabilidade garante que nenhuma decisão de modelação foi tomada de forma arbitrária, mas sim fundamentada nas necessidades reais do cliente.

2.3 Análise e Validação Geral dos Requisitos

Concluída a organização dos requisitos, a equipa procedeu a uma análise crítica e detalhada do conjunto obtido, com o objetivo de verificar a sua qualidade antes de o utilizar como base para a modelação conceptual. Esta validação incidu sobre três propriedades fundamentais: completude, consistência e não ambiguidade.

Completude. Os requisitos cobrem de forma abrangente todas as entidades centrais do domínio do EvenSync — eventos, sessões, organizadores, oradores, participantes, inscrições e pagamentos — estando diretamente alinhados com os objetivos enunciados na Secção 1.2. Não foi identificada nenhuma funcionalidade essencial em falta.

Consistência. As regras de controlo (RC1–RC6) foram definidas de forma complementar e coerente com os requisitos de descrição (RD1–RD9) e de manipulação (RM1–RM10), não existindo qualquer contradição entre eles. A título de exemplo, a restrição RC3 (unicidade de email) decorre naturalmente da existência dos atributos de email definidos em RD2, RD5 e RD7, reforçando a integridade do sistema sem criar conflitos.

Não Ambiguidade. Cada requisito foi redigido com linguagem precisa e objetiva. Sempre que as respostas do cliente apresentavam formulações vagas ou passíveis de interpretações múltiplas, a equipa procedeu à sua reformulação em conjunto com o cliente, garantindo que o significado final era único e inequívoco.

Validação com o Cliente. Por fim, o conjunto de requisitos foi submetido a uma validação final com o próprio António Carlos Silva Abelha, que confirmou que os requisitos levantados refletem fielmente as suas expectativas e necessidades para a plataforma EvenSync, podendo assim servir de alicerce à fase de modelação que se segue.

3 Modelação Conceptual

3.1 Apresentação da Abordagem de Modelação Realizada

Concluída a recolha e análise dos requisitos (Capítulo 2), a equipa iniciou o planeamento da estrutura da base de dados. A modelação conceptual teve como objetivo representar, de forma rigorosa e estruturada, o domínio do sistema EvenSync, servindo de ponte entre o catálogo de requisitos e as fases posteriores de modelação lógica e implementação física.

A abordagem seguida combinou uma perspetiva *top-down* com validações *bottom-up*. A partir dos Requisitos de Descrição (RD1–RD9) e da narrativa do caso de estudo, foram identificados os objetos com identidade própria e relevância operacional. Os Requisitos de Controlo (RC1–RC6) orientaram a definição de restrições de cardinalidade e regras de negócio. Os Requisitos de Manipulação (RM1–RM10) serviram como critério de validação, garantindo que cada consulta prevista é respondível através das entidades e relacionamentos modelados.

Desta análise resultaram **7 entidades principais**:

- **Evento** (RD1, RD3) — entidade central do sistema; representa as conferências, workshops e eventos corporativos a gerir;
- **Organizador** (RD2, RD3) — responsável pela gestão e supervisão de um ou vários eventos;
- **Sessão** (RD4) — atividade ou palestra que ocorre dentro de um determinado evento;
- **Orador** (RD5, RD6) — especialista ou convidado que apresenta numa ou mais sessões;
- **Participante** (RD7, RD8) — utilizador que se regista na plataforma e se inscreve em eventos;
- **Inscrição** (RD8, RC1, RC2) — registo formal da participação de um utilizador num evento;
- **Pagamento** (RD9, RC2) — transação financeira que valida e confirma uma inscrição.

O modelo conceptual foi construído recorrendo ao **Modelo Entidade-Relacionamento (ER)**, utilizando o **brModelo** para a produção do diagrama. Foram definidas entidades fortes, relacionamentos binários com cardinalidades explícitas (1:1, 1:N, N:M) e atributos simples e compostos, em conformidade com as regras de negócio observadas nos requisitos.

Assim, a equipa começou por identificar as entidades e os relacionamentos entre elas (Secções 3.2 e 3.3), caracterizou depois os atributos de cada entidade (Secção 3.4), e finalmente produziu o Diagrama ER completo do sistema (Secção 3.5).

3.2 Identificação e Caracterização das Entidades

A identificação das entidades resulta da análise dos Requisitos de Descrição (RD1–RD9) apresentados no Capítulo 2. Uma entidade representa um conjunto de objetos com existência independente que partilham propriedades comuns e necessitam de armazenamento persistente.

A Tabela 4 apresenta uma síntese das 7 entidades identificadas para o sistema EvenSync.

Entidade	Descrição	Requisitos
Evento	Conferência, workshop ou evento corporativo a gerir na plataforma	RD1, RD3
Organizador	Pessoa responsável pela gestão e supervisão de um ou vários eventos	RD2, RD3
Sessão	Atividade ou palestra que ocorre dentro de um determinado evento	RD4
Orador	Especialista ou convidado que apresenta numa ou mais sessões	RD5, RD6
Participante	Utilizador que se regista na plataforma e se inscreve em eventos	RD7, RD8
Inscrição	Registo formal da participação de um utilizador num evento	RD8, RC1, RC2
Pagamento	Transação financeira que valida e confirma uma inscrição	RD9, RC2

Table 4: Entidades do Sistema EvenSync

Nas subsecções seguintes apresenta-se a caracterização detalhada de cada entidade, incluindo justificação, atributos principais e relacionamentos.

3.2.1 Entidade: Evento

A entidade **Evento** foi identificada a partir de RD1 e RD3. Representa a entidade central do sistema EvenSync, correspondendo a qualquer conferência, workshop ou evento corporativo que o cliente António Carlos Silva Abelha pretende gerir. Possui existência independente e é referenciada por praticamente todas as outras entidades do sistema.

Atributos principais:

- **id_evento** (chave primária)
- **nome** (obrigatório — RD1)
- **tipo** (obrigatório — RD1: conferências, workshops, eventos corporativos, seminários — para garantir a integridade dos dados, evitando a entrada de tipos não padronizados que poderiam comprometer a consistência das consultas e relatórios do sistema)
- **capacidade** (obrigatório — RD1: número máximo de participantes)
- **descricao** (opcional — RD1)
- **data_inicio** e **data_fim** (obrigatórios — RD1)
- **localizacao** (obrigatório — RD1)

Relacionamentos principais:

- Um Evento é gerido por um ou mais Organizadores (N:M — RD3)
- Um Evento possui várias Sessões (1:N — RD4)
- Um Evento recebe várias Inscrições de Participantes (1:N — RD8)

3.2.2 Entidade: Organizador

A entidade **Organizador** foi identificada a partir de RD2 e RD3. Representa a pessoa ou entidade responsável pela criação e gestão operacional dos eventos na plataforma EvenSync.

Atributos principais:

- **id_organizador** (chave primária)
- **nome** (obrigatório — RD2)
- **email** (obrigatório, único — RD2, RC3)
- **telefone** (obrigatório — RD2)

Relacionamentos principais:

- Um Organizador pode gerir múltiplos Eventos (N:M — RD3)

3.2.3 Entidade: Sessão

A entidade **Sessão** foi identificada a partir de RD4. Representa uma atividade ou palestra específica que decorre dentro de um evento, com hora e sala próprias.

Atributos principais:

- **id_sessao** (chave primária)
- **tema** (obrigatório — RD4)
- **sala** (obrigatório — RD4, RC6)
- **data** (obrigatório — RD4)
- **hora_inicio** e **hora_fim** (obrigatórios — RD4, RC6)

Relacionamentos principais:

- Uma Sessão pertence a um único Evento (N:1 — RD4)
- Uma Sessão é apresentada por um ou mais Oradores (N:M — RD6)

3.2.4 Entidade: Orador

A entidade **Orador** foi identificada a partir de RD5 e RD6. Representa um especialista ou convidado responsável por apresentar o conteúdo de uma ou mais sessões do evento.

Atributos principais:

- **id_orador** (chave primária)
- **nome** (obrigatório — RD5)
- **email** (obrigatório, único — RD5, RC3)
- **especialidade** (obrigatório — RD5)

Relacionamentos principais:

- Um Orador pode apresentar múltiplas Sessões (N:M — RD6)

3.2.5 Entidade: Participante

A entidade **Participante** foi identificada a partir de RD7 e RD8. Representa qualquer pessoa que se regista na plataforma EvenSync com o intuito de participar em eventos.

Atributos principais:

- **id_participante** (chave primária)
- **nome** (obrigatório — RD7)
- **email** (obrigatório, único — RD7, RC3)
- **telemovel** (obrigatório — RD7)
- **organizacao** (opcional — RD7: instituição ou empresa de origem)

Relacionamentos principais:

- Um Participante pode realizar múltiplas Inscrições (1:N — RD8, RC1)

3.2.6 Entidade: Inscrição

A entidade **Inscrição** foi identificada a partir de RD8, RC1 e RC2. Representa o registo formal da intenção de participação de um Participante num Evento. Trata-se de uma entidade associativa, pois resulta do relacionamento entre Participante e Evento.

Atributos principais:

- **id_inscricao** (chave primária)
- **data_inscricao** (obrigatório — RD8)
- **estado** (obrigatório — RD8: confirmada, pendente ou cancelada)

Restrição crítica (RC1): Um Participante não pode ter mais do que uma inscrição ativa no mesmo Evento. Esta regra é garantida através de uma chave candidata composta por (**id_participante**, **id_evento**).

Relacionamentos principais:

- Uma Inscrição é realizada por um Participante (N:1)
- Uma Inscrição refere-se a um Evento (N:1)
- Uma Inscrição tem, no máximo, um Pagamento associado (1:1 — RC2)

3.2.7 Entidade: Pagamento

A entidade **Pagamento** foi identificada a partir de RD9 e RC2. Representa a transação financeira que confirma e valida uma inscrição num evento.

Atributos principais:

- **id_pagamento** (chave primária)
- **valor** (obrigatório — RD9)
- **data_pagamento** (obrigatório — RD9)
- **metodo** (obrigatório — RD9: cartão, transferência, MBWay, entre outros)
- **estado** (obrigatório — RD9: pago, pendente, rejeitado)

Restrição crítica (RC2): Cada Inscrição tem, no máximo, um Pagamento associado, garantindo a unicidade da transação por inscrição.

Relacionamentos principais:

- Um Pagamento está associado a uma única Inscrição (1:1 — RC2)

Esta identificação e caracterização das entidades estabelece a base para a definição dos relacionamentos (Secção 3.3) e dos atributos detalhados de cada entidade (Secção 3.4).

3.3 Identificação e Caracterização dos Relacionamentos

Nesta secção apresentam-se os relacionamentos identificados entre as 7 entidades do sistema EvenSync (Evento, Organizador, Sessão, Orador, Participante, Inscrição e Pagamento), incluindo o seu significado, cardinalidades, participação (total/parcial) e os requisitos de negócio que os originaram.

3.3.1 Visão Global dos Relacionamentos

A Tabela 5 apresenta a síntese dos 6 relacionamentos identificados no sistema:

ID	Relacionamento	Entidades	Card.	Partic.	Requisitos	Descrição
R1	realiza	Participante → Inscrição	1:N	P:T	RD8, RC1	Registo de inscrições.
R2	pertence	Inscrição → Evento	N:1	T:P	RD8	Associação de evento.
R3	precisa	Inscrição → Pagamento	1:1	P:T	RD9, RC2	Ligação de transação.
R4	gere	Organizador → Evento	N:M	T:T	RD3	Gestão do evento.
R5	tem	Evento → Sessão	1:N	T:T	RD4	Composição de evento.
R6	apresentada	Sessão → Orador	N:M	T:T	RD6	Atribuição de oradores.

Table 5: Síntese dos Relacionamentos do Sistema EvenSync

Os relacionamentos acima refletem a lógica de negócio descrita na secção de entidades. Nas subsecções seguintes, detalha-se cada um destes relacionamentos individualmente.

3.3.2 Caracterização Detalhada dos Relacionamentos

R1 — realiza (Participante — Inscrição)

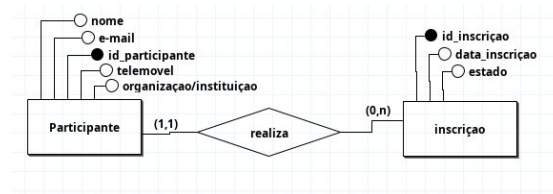


Figure 2: Relacionamento *realiza* entre Participante e Inscrição

O relacionamento *realiza* estabelece a ligação entre um participante e as inscrições que este efetua no sistema. Um participante pode realizar zero ou várias inscrições, refletindo o facto de que um utilizador registado no sistema pode nunca ter participado em nenhum evento, ou, em contrapartida, ter-se inscrito em múltiplos eventos ao longo do tempo. Do lado da inscrição, a participação é total e obrigatória: toda e qualquer inscrição tem de estar necessariamente associada a um participante, pois não faz sentido existir uma inscrição sem que haja alguém que a tenha efetuado. Este relacionamento é do tipo 1:N.

R2 — pertence (Inscrição — Evento)

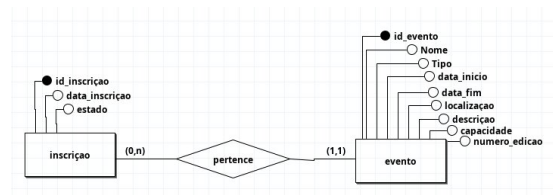


Figure 3: Relacionamento *pertence* entre Inscrição e Evento

O relacionamento *pertence* associa cada inscrição ao evento a que diz respeito. Cada inscrição pertence obrigatoriamente a um único evento, o que garante que não existem inscrições "solitas" no sistema, sem destino definido. Do lado do evento, a participação é parcial: um evento pode existir no sistema sem que nenhuma inscrição lhe tenha sido associada ainda, por exemplo, num período anterior à abertura das inscrições. Este relacionamento é também do tipo 1:N, onde um evento pode acumular várias inscrições ao longo do tempo.

R3 — precisa (Inscrição — Pagamento)

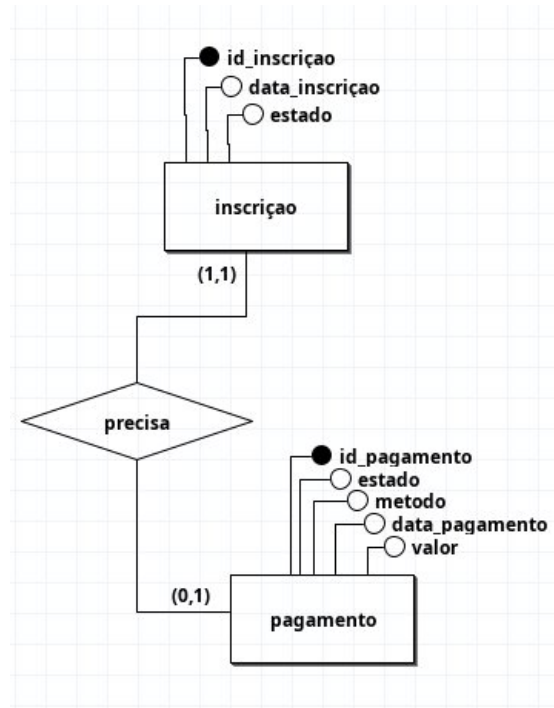


Figure 4: Relacionamento *precisa* entre Inscrição e Pagamento

O relacionamento *precisa* representa a ligação entre uma inscrição e o respetivo pagamento. Trata-se do único relacionamento do tipo 1:1 neste modelo, com participação parcial do lado da *Inscrição* e participação total do lado do *Pagamento* (P:T): uma inscrição pode existir sem que ainda tenha um pagamento associado (estado **pendente**), mas todo o pagamento está obrigatoriamente associado a uma inscrição específica. Esta assimetria reflete a regra de negócio em que o registo de inscrição é criado primeiro e o pagamento pode ser formalizado posteriormente, não podendo, no entanto, existir um pagamento sem uma inscrição correspondente.

R4 — gere (Organizador — Evento)

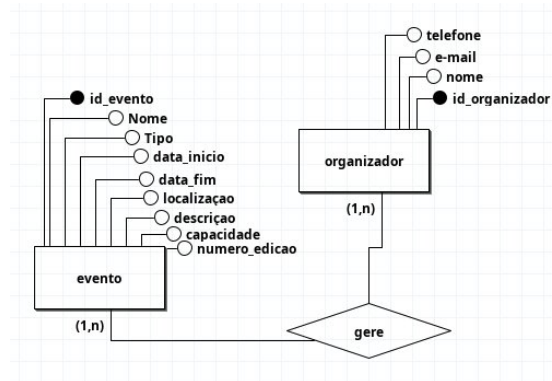


Figure 5: Relacionamento *gere* entre Organizador e Evento

O relacionamento *gere* traduz a responsabilidade que os organizadores têm sobre os eventos do sistema. Um organizador pode ser responsável por gerir um ou vários eventos, e um evento pode ser gerido por um ou vários organizadores em simultâneo. A participação é total em ambos os lados: todo o organizador tem de gerir pelo menos um evento, e todo o evento tem de ter pelo menos um organizador responsável. Por ser um relacionamento N:M com estas características, a sua implementação no modelo lógico exigirá a criação de uma tabela associativa.

R5 — tem (Evento — Sessão)

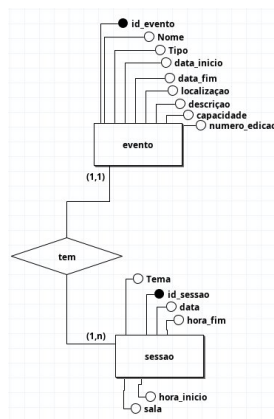


Figure 6: Relacionamento *tem* entre Evento e Sessão

O relacionamento *tem* expressa a composição de um evento em sessões. Um evento é constituído por uma ou várias sessões, e cada sessão pertence

obrigatoriamente a um único evento — conforme expresso em RD4, que define sessões como “associadas a um evento”. A participação é total em ambos os lados: um evento tem obrigatoriamente pelo menos uma sessão, e uma sessão existe sempre no âmbito de exatamente um evento. Por se tratar de um relacionamento 1:N, a sua implementação no modelo lógico é feita através de uma chave estrangeira *id_evento* na tabela *Sessao*, sem necessidade de tabela associativa.

R6 — apresentada (Sessão — Orador)

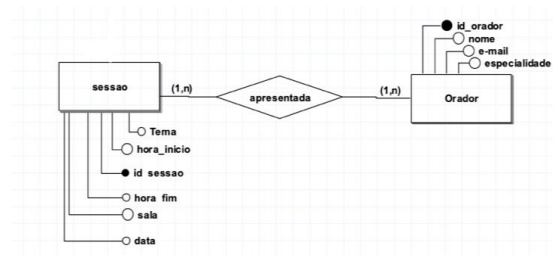


Figure 7: Relacionamento *apresentada* entre Sessão e Orador

O relacionamento *apresentada* liga as sessões aos oradores que nelas intervêm. Uma sessão é apresentada por um ou vários oradores, e um orador pode participar em uma ou várias sessões, refletindo a realidade de eventos onde os mesmos especialistas podem ser convidados a intervir em múltiplas ocasiões. A participação é total em ambos os lados: não existem sessões sem orador(es) atribuído(s), nem oradores registados no sistema que não estejam associados a pelo menos uma sessão. À semelhança dos relacionamentos anteriores de tipo N:M, a sua tradução para o modelo lógico implicará a criação de uma tabela associativa.

3.4 Identificação e Caracterização dos Atributos das Entidades e dos Relacionamentos

Nas subsecções seguintes caracterizam-se detalhadamente os atributos de cada uma das sete entidades do sistema. Quanto aos relacionamentos, os dois relacionamentos N:M identificados — *Gere* (Organizador–Evento) e *Apresentada* (Sessão–Orador) — não possuem atributos próprios para além das chaves estrangeiras que compõem as suas chaves primárias compostas, pelo que não é necessária uma caracterização adicional de atributos de relacionamento.

3.4.1 Entidade: Participante

A entidade *Participante* é caracterizada pelos dados pessoais e de contacto de cada pessoa registada no sistema. O atributo **id_participante** constitui a chave primária da entidade, sendo um identificador único gerado internamente pelo sistema que permite distinguir inequivocamente cada participante. O atributo **nome** regista o nome completo do participante, sendo de preenchimento obrigatório. O atributo **e-mail** armazena o endereço de correio eletrónico do participante, devendo ser único no sistema dado que serve também como meio de identificação e contacto. O atributo **telemovel** guarda o número de contacto telefónico, sendo de preenchimento obrigatório (RD7). Por fim, o atributo **organização/instituição** indica a entidade à qual o participante está afiliado, sendo igualmente opcional, pois nem todos os participantes representam necessariamente uma organização.

Atributo	Descrição	Domínio	Nulo	Exemplo	Req
id_participante	Identificador sequencial	INT	N	1	—
nome	Nome completo do participante	VARCHAR(100)	N	Tiago Santos	RD1
e-mail	Endereço eletrónico	VARCHAR(100)	N	tiago@email.com	RD1
telemovel	Número de telemóvel	VARCHAR(15)	N	912345678	RD7
organização	Instituição que representa	VARCHAR(100)	S	UMinho	RD1

Table 6: Atributos da entidade Participante

Restrições:

- ▷ id_participante PRIMARY KEY
- ▷ e-mail UNIQUE

3.4.2 Entidade: Inscrição

A entidade *Inscrição* regista os dados relativos ao ato de inscrição de um participante num evento. O atributo **id_inscrição** é a chave primária, identificando de forma única cada registo de inscrição no sistema. O atributo **data_inscrição** armazena a data em que a inscrição foi efetuada, sendo obrigatório para fins de controlo e rastreabilidade. O atributo **estado** indica a situação atual da inscrição (por exemplo: pendente, confirmada ou cancelada), sendo fundamental para a gestão do processo.

Atributo	Descrição	Domínio	Nulo	Exemplo	Req
id_inscrição	Identificador sequencial	INT	N	100	—
data_inscrição	Data do registo	DATETIME	N	2026-05-10 10:00	RD8
estado	Situação atual	VARCHAR(15)	N	confirmada	RC1

Table 7: Atributos da entidade Inscrição

Restrições:

▷ **id_inscrição** PRIMARY KEY

▷ **estado** IN {'pendente', 'confirmada', 'cancelada'}

3.4.3 Entidade: Pagamento

A entidade *Pagamento* armazena toda a informação relativa ao pagamento associado a uma inscrição. O atributo **id_pagamento** é a chave primária que identifica univocamente cada registo de pagamento. O atributo **estado** indica se o pagamento foi concluído, está pendente ou foi rejeitado. O atributo **metodo** especifica a forma de pagamento utilizada (por exemplo: cartão de crédito, transferência bancária ou MB Way). O atributo **data_pagamento** regista a data em que o pagamento foi processado. O atributo **valor** indica o montante pago.

Atributo	Descrição	Domínio	Nulo	Exemplo	Req
id_pagamento	Identificador sequencial	INT	N	200	—
estado	Ponto de situação	VARCHAR(15)	N	pago	RD9
metodo	Forma de pagamento	VARCHAR(20)	N	MBWay	RD9
data_pagamento	Data de processamento	DATETIME	N	2026-05-10 10:05	RD9
valor	Montante em euros	DECIMAL(8,2)	N	25.50	RD9

Table 8: Atributos da entidade Pagamento

Restrições:

- ▷ id_pagamento PRIMARY KEY
- ▷ estado IN {'pago', 'pendente', 'rejeitado'}
- ▷ metodo IN {'cartão', 'transferência', 'MBWay'}

3.4.4 Entidade: Evento

A entidade *Evento* é a entidade central do modelo, agregando a informação descritiva de cada evento gerido pelo sistema. O atributo **id_evento** é a chave primária. O atributo **Nome** identifica o evento de forma textual. O atributo **Tipo** classifica o evento segundo a sua natureza (conferências, workshops, eventos corporativos, seminários), para garantir a integridade dos dados ao restringir os valores permitidos e prevenir inconsistências que poderiam afetar a análise estatística e a geração de relatórios precisos. O atributo **numero_edicao** identifica a edição do evento (por exemplo: 1.^a, 2.^a edição). Os atributos **data_inicio** e **data_fim** delimitam o período de realização do evento. O atributo **localização** indica o local onde o evento decorre. O atributo **descrição** permite registar uma apresentação mais detalhada do evento. Por último, o atributo **capacidade** define o número máximo de participantes admitidos, sendo relevante para o controlo das inscrições.

Atributo	Descrição	Domínio	Nulo	Exemplo	Req
id_evento	Identificador sequencial	INT	N	5	—
Nome	Nome do evento	VARCHAR(150)	N	FestivaliUM	RD1
Tipo	Categoria do evento	VARCHAR(50)	N	Conferência	RD1
numero_edicao	Número de edição	INT	N	3	RD1
data_inicio	Data de início	DATE	N	2026-06-01	RD1
data_fim	Data de término	DATE	N	2026-06-05	RD1
localização	Sede principal	VARCHAR(150)	N	Braga	RD1
descrição	Detalhes do evento	TEXT	S	O maior evento...	RD1
capacidade	Lotação máxima	INT	N	500	RD1

Table 9: Atributos da entidade Evento

Restrições:

- ▷ **id_evento** PRIMARY KEY
- ▷ **data_inicio** ≤ **data_fim**

3.4.5 Entidade: Organizador

A entidade *Organizador* representa as pessoas responsáveis pela gestão dos eventos. O atributo `id_organizador` é a chave primária e identificador único de cada organizador no sistema. O atributo `nome` regista o nome completo do organizador. O atributo `e-mail` armazena o endereço de correio eletrónico, sendo um meio privilegiado de contacto e devendo ser único. O atributo `telefone` guarda o número de contacto telefónico do organizador, de preenchimento opcional.

Atributo	Descrição	Domínio	Nulo	Exemplo	Req
<code>id_organizador</code>	Identificador sequencial	INT	N	10	—
<code>nome</code>	Nome do responsável	VARCHAR(100)	N	Maria Silva	RD2
<code>e-mail</code>	Endereço eletrônico	VARCHAR(100)	N	maria@org.pt	RD2
<code>telefone</code>	Número de contacto	VARCHAR(15)	S	919876543	RD2

Table 10: Atributos da entidade Organizador

Restrições:

- ▷ `id_organizador` PRIMARY KEY
- ▷ `e-mail` UNIQUE

3.4.6 Entidade: Sessão

A entidade *Sessão* representa cada sessão individual que compõe um evento. O atributo `id_sessao` é a chave primária. O atributo `Tema` descreve o tópico ou assunto abordado na sessão. O atributo `data` indica o dia em que a sessão decorre. Os atributos `hora_inicio` e `hora_fim` delimitam o intervalo temporal da sessão dentro do dia. Por fim, o atributo `sala` identifica o espaço físico onde a sessão tem lugar, sendo relevante para a gestão logística do evento.

Atributo	Descrição	Domínio	Nulo	Exemplo	Req
<code>id_sessao</code>	Identificador sequencial	INT	N	50	—
<code>Tema</code>	Tópico a abordar	VARCHAR(150)	N	IA no Futuro	RD4
<code>data</code>	Data de realização	DATE	N	2026-06-02	RD4
<code>hora_inicio</code>	Início da sessão	TIME	N	14:00	RD4
<code>hora_fim</code>	Fim da sessão	TIME	N	16:00	RD4
<code>sala</code>	Espaço físico alocado	VARCHAR(50)	N	Auditório B1	RD4

Table 11: Atributos da entidade Sessão

Restrições:

▷ `id_sessao` PRIMARY KEY

3.4.7 Entidade: Orador

A entidade *Orador* regista os dados dos especialistas que apresentam as sessões. O atributo `id_orador` é a chave primária, identificando univocamente cada orador no sistema. O atributo `nome` armazena o nome completo do orador. O atributo `e-mail` guarda o contacto eletrónico, sendo também potencialmente único. O atributo `especialidade` descreve a área de conhecimento ou domínio de atuação do orador, sendo relevante para a organização e divulgação do evento.

Atributo	Descrição	Domínio	Nulo	Exemplo	Req
<code>id_orador</code>	Identificador sequencial	INT	N	20	—
<code>nome</code>	Nome completo	VARCHAR(100)	N	João Oliveira	RD5
<code>e-mail</code>	Endereço eletrónico	VARCHAR(100)	N	joao@email.pt	RD5
<code>especialidade</code>	Área de atuação	VARCHAR(100)	N	Cibersegurança	RD5

Table 12: Atributos da entidade Orador

Restrições:

- ▷ `id_orador` PRIMARY KEY
- ▷ `e-mail` UNIQUE

Os identificadores utilizam `INT` com `AUTO_INCREMENT`, garantindo unicidade automática. Para texto com limite previsível empregou-se `VARCHAR` com dimensão ajustada ao contexto (15 para telefones, 100 para nomes e e-mails, 150 para temas e nomes de evento). Para descrições sem limite fixo usou-se `TEXT`. Datas são representadas com `DATE` e horários de sessão com `TIME`. O valor do pagamento usa `DECIMAL(8,2)`, assegurando precisão monetária. Para atributos com conjunto fixo de valores — como `tipo` do Evento, `estado` da Inscrição e `metodo` do Pagamento — aplicou-se `VARCHAR` com restrição `CHECK`, impedindo a inserção de valores inválidos ao nível da base de dados.

3.5 Apresentação e Explicação do Diagrama ER Produzido

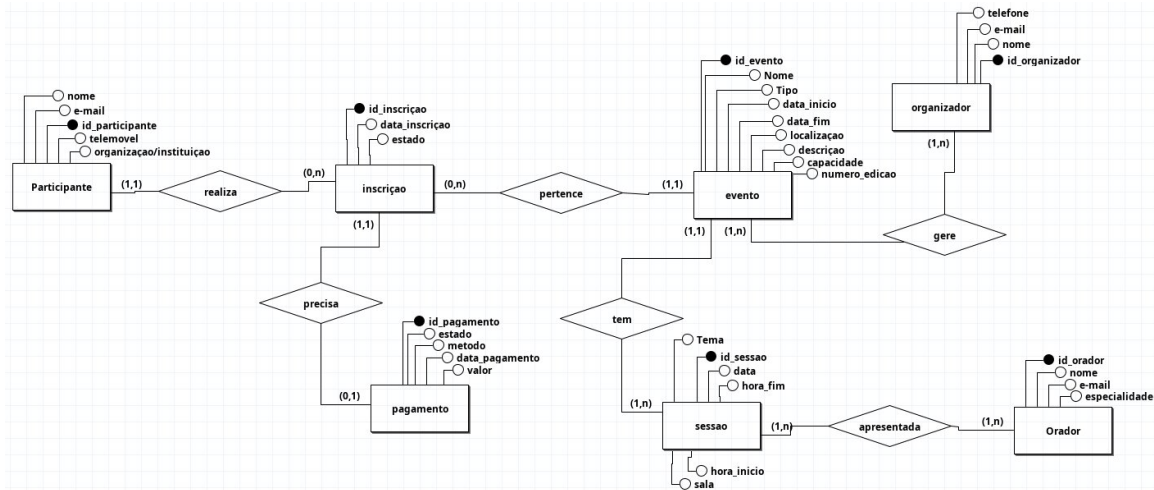


Figure 8: Diagrama Entidade-Relacionamento do sistema EvenSync

O diagrama Entidade-Relacionamento (ER) produzido pelo grupo encontra-se representado na Figura 8 e materializa visualmente a totalidade das entidades, atributos e relacionamentos identificados e caracterizados nas secções anteriores. O diagrama foi construído recorrendo à notação de Chen, onde as entidades são representadas por retângulos, os relacionamentos por losangos, e os atributos por elipses — sendo os atributos chave primária distinguidos por uma elipse preenchida.

O modelo é composto por sete entidades: *Participante*, *Inscrição*, *Pagamento*, *Evento*, *Organizador*, *Sessão* e *Orador*, interligadas através de seis relacionamentos que asseguram a integridade e coerência do sistema.

A entidade central do modelo é *Evento*, que estabelece ligações diretas com quatro outras entidades. Por um lado, associa-se à entidade *Inscrição* através do relacionamento *pertence*, que regista a adesão de participantes ao evento. Por outro, relaciona-se com *Organizador* através de *gere*, refletindo a responsabilidade de gestão do evento, e com *Sessão* através de *tem*, representando a decomposição do evento nas suas atividades individuais.

A entidade *Participante* liga-se à entidade *Inscrição* pelo relacionamento *realiza*, formalizando o ato de um participante se inscrever num evento. A entidade *Inscrição* estabelece ainda uma ligação obrigatória e exclusiva com a entidade *Pagamento* através do relacionamento *precisa*, refletindo a regra de negócio de que toda a inscrição requer um pagamento associado. Por fim, a entidade *Sessão* associa-se à entidade *Orador* pelo relacionamento *apresentada*, indicando quais os oradores responsáveis por cada sessão.

4 Modelação Lógica

4.1 Construção e Validação do Modelo de Dados Lógico

A modelação lógica tem como objetivo traduzir o modelo conceptual para um modelo relacional, mantendo a coerência com os Requisitos de Descrição (RD), Controlo (RC) e Manipulação (RM) definidos no Capítulo 2.

4.1.1 Abordagem de Construção do Modelo Lógico

O modelo de dados lógico foi construído a partir do Diagrama Entidade-Relacionamento desenvolvido na fase de modelação conceptual (Secção 3.5). A conversão seguiu as regras clássicas de transformação do modelo ER para o modelo lógico relacional. Em termos gerais, foram adotados os seguintes princípios:

- Cada entidade do modelo conceptual foi convertida numa tabela, preservando os seus atributos e identificador único;
- Os identificadores das entidades passaram a corresponder a chaves primárias (*Primary Keys*);
- Os relacionamentos do tipo 1:N foram implementados através da introdução de chaves estrangeiras (*Foreign Keys*) na tabela do lado N;
- Os relacionamentos do tipo 1:1 foram implementados através da introdução de uma chave estrangeira com restrição **UNIQUE** no lado de participação total;
- Os relacionamentos do tipo N:M deram origem a tabelas associativas, cuja chave primária composta é formada pelas chaves primárias das entidades participantes;
- As regras de negócio foram materializadas através de restrições de integridade (**UNIQUE**, **CHECK**, **NOT NULL**).

A construção do modelo lógico foi realizada manualmente pela equipa, com apoio da ferramenta MySQL Workbench para validação da coerência estrutural do esquema (Figura 9).

4.1.2 Estrutura Geral do Modelo Lógico

Como resultado do processo descrito, obteve-se um modelo lógico constituído por **nove tabelas**: sete correspondentes às entidades identificadas no modelo conceptual e duas tabelas associativas resultantes dos relacionamentos N:M.

Tabelas de entidades:

- **Participante** — utilizadores registados na plataforma
- **Evento** — eventos a gerir no sistema
- **Organizador** — responsáveis pela gestão dos eventos
- **Sessao** — atividades individuais que compõem um evento
- **Orador** — especialistas que apresentam sessões
- **Inscricao** — registos de participação em eventos
- **Pagamento** — transações financeiras associadas a inscrições

Tabelas associativas:

- **Gere** — materializa o relacionamento N:M entre Organizador e Evento
- **Apresentada** — materializa o relacionamento N:M entre Sessao e Orador

4.1.3 Validação do Modelo de Dados Lógico

A validação do modelo lógico foi realizada com base em três critérios principais:

- **Coerência com o modelo conceptual:** confirmou-se que todas as entidades, relacionamentos e regras definidas no diagrama ER estão corretamente representadas no esquema relacional, nomeadamente as cardinalidades e as participações totais/parciais;
- **Cobertura dos requisitos:** verificou-se que o modelo lógico suporta todos os Requisitos de Descrição (RD1–RD9) e de Controlo (RC1–RC6), nomeadamente o registo de eventos, sessões, oradores, participantes, inscrições e pagamentos, bem como as regras de unicidade e integridade referencial;
- **Suporte às interrogações do utilizador (RMs):** analisou-se se o esquema relacional permite formular todas as interrogações definidas na Secção 2.2 (RM1–RM10), sem necessidade de informação adicional ou redundante.

O resultado desta validação indica que o modelo de dados lógico é consistente, completo e adequado aos objetivos do sistema EvenSync, encontrando-se preparado para as fases seguintes.

4.2 Apresentação e Explicação do Modelo Lógico Produzido

Para a construção do esquema lógico, a equipa iniciou o processo de derivação de relações a partir do diagrama conceptual. Os atributos simples de cada entidade foram mapeados diretamente para atributos das respetivas relações. Procedeu-se de seguida ao mapeamento dos relacionamentos: conforme a cardinalidade de cada um, foi adicionado um atributo de chave estrangeira na tabela do lado N (relacionamentos 1:N e 1:1) ou criada uma nova tabela associativa (relacionamentos N:M).

O diagrama do modelo lógico produzido encontra-se representado na Figura 9. Nas subsecções seguintes descreve-se individualmente o processo de criação de cada relação.

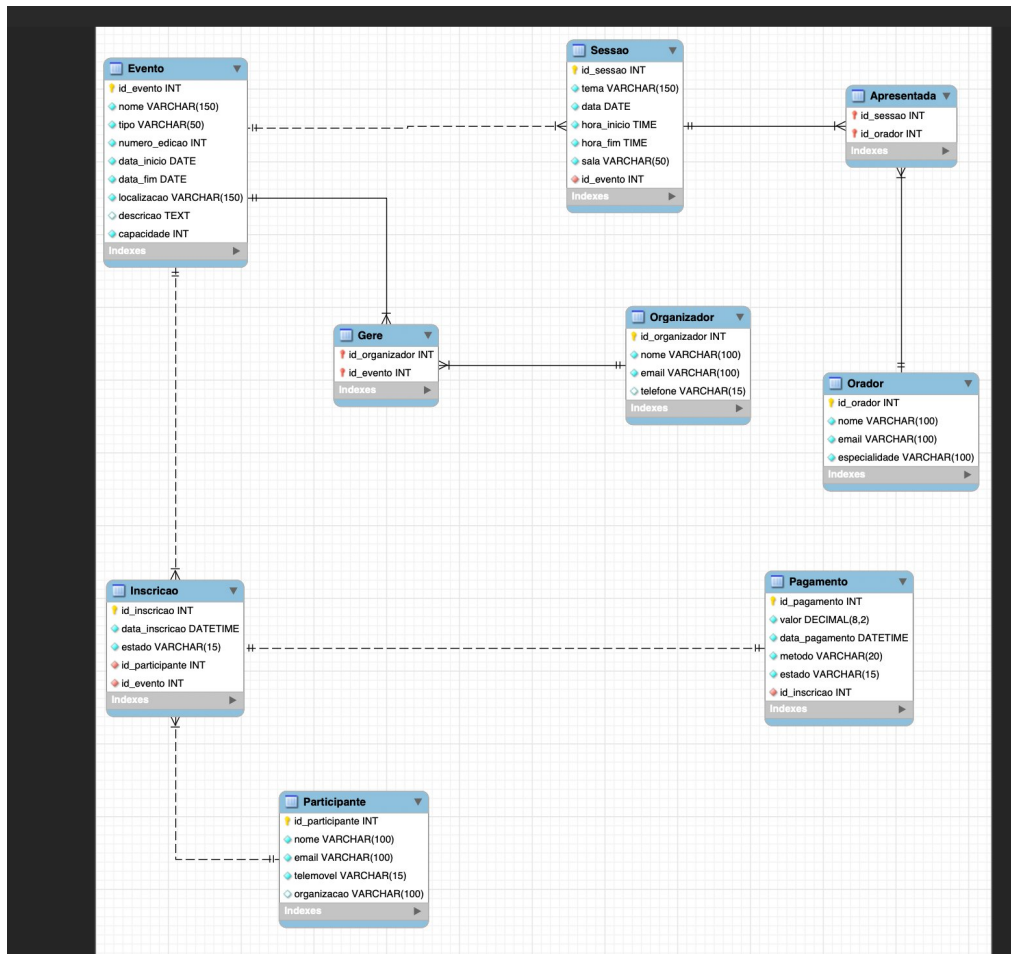


Figure 9: Diagrama do Modelo Lógico do sistema EvenSync (gerado com MySQL Workbench)

4.2.1 Relação: Participante

A relação *Participante* deriva diretamente da entidade homônima do modelo conceptual (RD7). Os seus atributos simples foram mapeados para colunas da tabela. Foram identificadas duas chaves candidatas: **id_participante** e **email** — ambas identificam univocamente um participante. Optou-se por **id_participante** como chave primária por ser controlada internamente pelo sistema. O atributo **email** mantém a restrição **UNIQUE** (RC3).

Atributo	Domínio	Nulo	Chave Estrangeira
id_participante	INT	N	N
nome	VARCHAR(100)	N	N
email	VARCHAR(100)	N	N
telemovel	VARCHAR(15)	N	N
organizacao	VARCHAR(100)	S	N

Table 13: Dicionário de dados da relação Participante

4.2.2 Relação: Evento

A relação *Evento* é a tabela central do sistema, derivando da entidade homônima (RD1). Agrega toda a informação descritiva de cada evento. A chave primária *id_evento* é gerada automaticamente pelo sistema. A restrição *CHECK(data_inicio <= data_fim)* garante a coerência temporal do evento. O atributo *numero_edicao* registra a edição do evento, sendo de preenchimento obrigatório.

Atributo	Domínio	Nulo	Chave Estrangeira
id_evento	INT	N	N
nome	VARCHAR(150)	N	N
tipo	VARCHAR(50)	N	N
numero_edicao	INT	N	N
data_inicio	DATE	N	N
data_fim	DATE	N	N
localizacao	VARCHAR(150)	N	N
descricao	TEXT	S	N
capacidade	INT	N	N

Table 14: Dicionário de dados da relação Evento

4.2.3 Relação: Organizador

A relação *Organizador* deriva da entidade homônima (RD2). Representa as pessoas responsáveis pela gestão dos eventos. Tal como em *Participante*, existem duas chaves candidatas (*id_organizador* e *email*), sendo *id_organizador* escolhida como chave primária e *email* mantido como *UNIQUE* (RC3).

Atributo	Domínio	Nulo	Chave Estrangeira
id_organizador	INT	N	N
nome	VARCHAR(100)	N	N
email	VARCHAR(100)	N	N
telefone	VARCHAR(15)	N	N

Table 15: Dicionário de dados da relação Organizador

4.2.4 Relação: Sessao

A relação *Sessao* deriva da entidade homônima (RD4) e representa cada atividade individual que compõe um evento. A chave primária é **id_sessao**. O relacionamento 1:N *tem* (R5) é implementado através da chave estrangeira **id_evento**, que referencia *Evento* e garante que cada sessão pertence a exatamente um evento. A restrição **CHECK(hora_inicio < hora_fim)** garante a validade do intervalo temporal, necessária para suportar a regra de negócio RC6 (não sobreposição de horários).

Atributo	Domínio	Nulo	Chave Estrangeira
id_sessao	INT	N	N
tema	VARCHAR(150)	N	N
data	DATE	N	N
hora_inicio	TIME	N	N
hora_fim	TIME	N	N
sala	VARCHAR(50)	N	N
id_evento	INT	N	S

Table 16: Dicionário de dados da relação Sessao

Restrições: id_evento FK → Evento(id_evento)

4.2.5 Relação: Orador

A relação *Orador* deriva da entidade homônima (RD5) e registra os especialistas que apresentam sessões. A estrutura é análoga à de *Participante*: **id_orador** como chave primária e **email** com restrição **UNIQUE** (RC3).

Atributo	Domínio	Nulo	Chave Estrangeira
id_orador	INT	N	N
nome	VARCHAR(100)	N	N
email	VARCHAR(100)	N	N
especialidade	VARCHAR(100)	N	N

Table 17: Dicionário de dados da relação Orador

4.2.6 Relação: Inscricao

A relação *Inscricao* deriva da entidade homónima (RD8) e incorpora duas chaves estrangeiras: *id_participante* que referencia *Participante* (R1) e *id_evento* que referencia *Evento* (R2). A chave candidata composta (*id_participante*, *id_evento*) é definida como **UNIQUE**, garantindo que um participante não se inscreve mais do que uma vez no mesmo evento (RC1). O atributo *estado* regista a situação da inscrição com os valores possíveis: 'pendente', 'confirmada' ou 'cancelada'.

Atributo	Domínio	Nulo	Chave Estrangeira
id_inscricao	INT	N	N
data_inscricao	DATETIME	N	N
estado	VARCHAR(15)	N	N
id_participante	INT	N	S
id_evento	INT	N	S

Table 18: Dicionário de dados da relação Inscricao

4.2.7 Relação: Pagamento

A relação *Pagamento* deriva da entidade homónima (RD9) e materializa o relacionamento 1:1 *precisa* entre *Inscricao* e *Pagamento*. Incorpora a chave estrangeira *id_inscricao* com restrição **UNIQUE**, garantindo que cada inscrição tem no máximo um pagamento associado (RC2). Os atributos *metodo* e *estado* são limitados a conjuntos fixos de valores através de restrições **CHECK**.

Atributo	Domínio	Nulo	Chave Estrangeira
id_pagamento	INT	N	N
valor	DECIMAL(8,2)	N	N
data_pagamento	DATETIME	N	N
metodo	VARCHAR(20)	N	N
estado	VARCHAR(15)	N	N
id_inscricao	INT	N	S

Table 19: Dicionário de dados da relação Pagamento

4.2.8 Relação: Gere

A relação *Gere* é uma tabela associativa criada para materializar o relacionamento N:M *gere* entre *Organizador* e *Evento* (R4, RD3). A chave primária composta é formada pelos dois atributos, que funcionam também como chaves estrangeiras.

Atributo	Domínio	Nulo	Chave Estrangeira
id_organizador	INT	N	S
id_evento	INT	N	S

Table 20: Dicionário de dados da relação Gere

4.2.9 Relação: Apresentada

A relação *Apresentada* é uma tabela associativa que materializa o relacionamento N:M *apresentada* entre *Sessao* e *Orador* (R6, RD6), permitindo que uma sessão tenha múltiplos oradores e que um orador apresente em múltiplas sessões.

Atributo	Domínio	Nulo	Chave Estrangeira
id_sessao	INT	N	S
id_orador	INT	N	S

Table 21: Dicionário de dados da relação Apresentada

4.3 Normalização de Dados

O objetivo desta secção é justificar que o modelo lógico relacional definido — *Participante*, *Evento*, *Organizador*, *Sessao*, *Orador*, *Inscricao*, *Pagamento*, *Gere* e *Apresentada* — se encontra normalizado, reduzindo a redundância e prevenindo anomalias de inserção, atualização e remoção.

A normalização foi verificada com base na identificação das dependências funcionais (DFs), das chaves primárias e alternativas, e na análise das formas normais (1FN, 2FN e 3FN) para cada relação do esquema final.

4.3.1 Dependências Funcionais e Chaves Identificadas

A partir do esquema relacional e das restrições definidas (PK, UNIQUE, FK), identificam-se as seguintes DFs principais:

Participante (id_participante, nome, email, telemovel, organizacao)

- id_participante → nome, email, telemovel, organizacao
- email → id_participante, nome, telemovel, organizacao (*chave alternativa — UNIQUE*)

Evento (id_evento, nome, tipo, numero_edicao, data_inicio, data_fim, localizacao, descricao, capacidade)

- id_evento → nome, tipo, numero_edicao, data_inicio, data_fim, localizacao, descricao, capacidade

Organizador (id_organizador, nome, email, telefone)

- id_organizador → nome, email, telefone
- email → id_organizador, nome, telefone (*chave alternativa — UNIQUE*)

Sessao (id_sessao, tema, data, hora_inicio, hora_fim, sala, id_evento)

- id_sessao → tema, data, hora_inicio, hora_fim, sala, id_evento

Orador (id_orador, nome, email, especialidade)

- id_orador → nome, email, especialidade
- email → id_orador, nome, especialidade (*chave alternativa — UNIQUE*)

Inscricao (id_inscricao, data_inscricao, estado, id_participante, id_evento)

- id_inscricao → data_inscricao, estado, id_participante, id_evento
- (id_participante, id_evento) → id_inscricao, data_inscricao, estado (*chave alternativa — UNIQUE, RC1*)

Pagamento (id_pagamento, valor, data_pagamento, metodo, estado, id_inscricao)

- id_pagamento → valor, data_pagamento, metodo, estado, id_inscricao
- id_inscricao → id_pagamento, valor, data_pagamento, metodo, estado (*chave alternativa — UNIQUE, RC2*)

Gere, Apresentada — apenas atributos de chave; não existem DFs adicionais.

4.3.2 Verificação das Formas Normais

A) Primeira Forma Normal (1FN)

Todas as relações respeitam a 1FN:

- Os atributos são atômicos — não existem listas nem valores repetidos num mesmo campo;

- Cada tabela possui uma chave primária bem definida;
- Os relacionamentos N:M foram decompostos em tabelas associativas (*Gere*, *Apresentada*), eliminando grupos repetitivos.

Todas as relações estão em **1FN**. ✓

B) Segunda Forma Normal (2FN)

A 2FN é relevante quando existem chaves compostas e dependências parciais:

- As relações *Participante*, *Evento*, *Organizador*, *Sessao*, *Orador*, *Inscricao* e *Pagamento* possuem chave primária simples, pelo que estão trivialmente em 2FN;
- As relações *Gere* e *Apresentada* possuem chave primária composta, mas não contêm atributos não-chave — logo não podem existir dependências parciais.

Todas as relações estão em **2FN**. ✓

C) Terceira Forma Normal (3FN)

Em 3FN não podem existir dependências transitivas (chave $\rightarrow$ A $\rightarrow$ B, onde B é não-chave):

- *Inscricao* não guarda atributos de *Participante* (nome, email) nem de *Evento* (nome, tipo); guarda apenas as FKs (*id_participante*, *id_evento*), evitando transitividades e redundância;
- *Pagamento* guarda apenas a referência à inscrição (*id_inscricao*), não duplicando dados como o nome do participante ou o nome do evento;
- Nas restantes relações, todos os atributos não-chave dependem diretamente da chave primária e não entre si.

Todas as relações estão em **3FN**. ✓

4.3.3 Anomalias Evitadas

A normalização até 3FN elimina as seguintes anomalias do modelo:

- **Inserção:** é possível registar um *Evento* sem inscrições associadas, e um *Participante* sem ter ainda realizado qualquer inscrição, sem violar a integridade do sistema;
- **Atualização:** alterar dados de um *Participante* (por exemplo, o email ou o nome) não implica atualizar registos em *Inscricao* ou *Pagamento*, pois esses atributos não são replicados — apenas a FK *id_participante* está presente;

- **Remoção:** eliminar um *Evento* não remove os dados do *Organizador* nem do *Participante*; os registos dependentes (*Inscricao*, *Pagamento*) são geridos de forma controlada pelas regras de integridade referencial (ON DELETE RESTRICT).

4.3.4 Conclusão

O modelo lógico relacional do EvenSync encontra-se normalizado até à 3FN, garantindo integridade, consistência e eficiência nas operações de manipulação de dados. A estrutura está preparada para a fase de implementação física (Capítulo 5).

4.4 Validação do Modelo com Interrogações do Utilizador

A equipa validou o modelo lógico produzido através da implementação das interrogações definidas pelos Requisitos de Manipulação (RM1–RM10) identificados no Capítulo 2. Para assegurar a correção do raciocínio e testar as interrogações antes da implementação em SQL, foi utilizada a calculadora de álgebra relacional **RelaX**.

Na notação utilizada: σ representa a selecção, π a projecção, $\bowtie$ a junção natural e $\mathcal{G}$ a operação de agrupamento com função de agregação. Os parâmetros variáveis são indicados por letras maiúsculas (X , T , etc.).

RM1 — Consultar todos os eventos, com filtro por tipo T :

$$\pi_{\text{id_evento}, \text{nome}, \text{tipo}, \text{data_inicio}, \text{data_fim}, \text{localizacao}} (\sigma_{\text{tipo} = T}(\text{Evento}))$$

Para listar todos os eventos, omite-se a selecção σ , retendo apenas a projecção sobre os atributos desejados.

RM2 — Programa completo do evento X (sessões, oradores, salas e horários):

$$\pi_{S.\text{tema}, S.\text{sala}, S.\text{data}, S.\text{hora_inicio}, S.\text{hora_fim}, O.\text{nome}} (\sigma_{E.\text{id_evento} = X} (E \bowtie S \bowtie \text{Apresentada} \bowtie O))$$

onde $E = \text{Evento}$, $S = \text{Sessao}$, $O = \text{Orador}$.

RM3 — Lista de inscritos no evento X com o estado da inscrição:

$$\pi_{P.\text{nome}, P.\text{email}, I.\text{estado}} (\sigma_{I.\text{id_evento} = X} (P \bowtie \text{Inscricao}))$$

onde $P = \text{Participante}$, $I = \text{Inscricao}$.

RM4 — Histórico de inscrições do participante X :

$$\pi_{E.\text{nome}, I.\text{data_inscricao}, I.\text{estado}} (\sigma_{P.\text{id_participante} = X} (P \bowtie I \bowtie E))$$

onde $P = Participante$, $I = Inscricao$, $E = Evento$.

RM5 — Total de receitas do evento X (pagamentos com estado 'pago'):

$$\mathcal{G}_{SUM(Pg.valor)}(\pi_{Pg.valor}(\sigma_{I.id_evento=X \wedge Pg.estado='pago'}(Inscricao \bowtie Pagamento)))$$

onde $I = Inscricao$, $Pg = Pagamento$.

RM6 — Eventos com maior taxa de ocupação (inscrições confirmadas / capacidade):

Definindo a relação auxiliar de contagem de inscrições confirmadas:

$$Contagem \leftarrow \rho_{(id_evento, n_confirmadas)}(\mathcal{G}_{id_evento; COUNT(*)}(\sigma_{estado='confirmada'}(Inscricao)))$$

$$\pi_{E.nome, C.n_confirmadas, E.capacidade}(Evento \bowtie_{id_evento} Contagem)$$

Ordenando pelo rácio $n_confirmadas/capacidade$ de forma decrescente, obtêm-se os eventos com maior taxa de ocupação.

RM7 — Oradores que apresentam sessões do evento X :

$$\pi_{O.nome, O.especialidade}(\sigma_{E.id_evento=X}(E \bowtie S \bowtie Apresentada \bowtie O))$$

onde $E = Evento$, $S = Sessao$, $O = Orador$.

RM8 — Consultar sessões por sala ou intervalo de datas:

$$\pi_{S.tema, S.sala, S.data, S.hora_inicio, S.hora_fim}(\sigma_{(S.sala=R) \vee (S.data \geq D_{ini} \wedge S.data \leq D_{fim})}(Sessao))$$

onde R é a sala e D_{ini}/D_{fim} definem o intervalo de datas.

RM9 — Oradores de uma sessão Y :

$$\pi_{O.nome, O.especialidade}(\sigma_{S.id_sessao=Y}(Sessao \bowtie Apresentada \bowtie Orador))$$

onde $S = Sessao$ e $O = Orador$.

RM10 — Pagamentos pendentes ou rejeitados:

$$\pi_{P.nome, E.nome, Pg.valor, Pg.estado}(\sigma_{Pg.estado \in \{'pendente', 'rejeitado'\}}(Pagamento \bowtie Inscricao \bowtie Participante \bowtie E))$$

onde $P = Participante$, $Pg = Pagamento$ e $E = Evento$.

A expressibilidade de todos os dez requisitos de manipulação em álgebra relacional, usando exclusivamente as relações e atributos do modelo lógico, confirma que o modelo produzido é **completo e válido** face ao conjunto de necessidades identificadas pelo utilizador.

5 Implementação Física

A fase de implementação física consistiu na concretização do modelo lógico validado num Sistema de Gestão de Bases de Dados real. O SGBD escolhido foi o **MySQL 8.x**, devido à sua maturidade, ampla documentação, suporte a restrições de integridade referencial, e compatibilidade com as ferramentas utilizadas ao longo do projeto. O esquema físico foi criado na base de dados denominada **eventsync**, com codificação **utf8mb4** e collation **utf8mb4_unicode_ci**, garantindo suporte a caracteres acentuados e especiais.

5.1 Apresentação e Explicação da Base de Dados Implementada

O modelo físico do EvenSync é composto por **nove tabelas**, que correspondem diretamente às entidades e relacionamentos N:M identificados nas fases de modelação. A tabela 22 apresenta uma visão geral das tabelas criadas e a sua correspondência com os requisitos de descrição.

Tabela	Descrição	RD
Participante	Dados dos participantes inscritos nos eventos	RD7
Evento	Informação sobre os eventos organizados	RD1
Organizador	Dados dos organizadores dos eventos	RD2
Sessao	Sessões associadas a cada evento	RD4
Orador	Dados dos oradores convidados	RD5
Inscricao	Registo das inscrições de participantes em eventos	RD8
Pagamento	Pagamentos associados a inscrições	RD9
Gere	Associação N:M entre Organizador e Evento	RD3
Apresentada	Associação N:M entre Sessao e Orador	RD6

Table 22: Tabelas do modelo físico do EvenSync

A integridade dos dados é assegurada por múltiplos mecanismos complementares:

- **Chaves primárias** com **AUTO_INCREMENT** em todas as tabelas principais;
- **Chaves estrangeiras** com as cláusulas **ON UPDATE CASCADE** e **ON DELETE** adequadas a cada relação;
- **Restrições UNIQUE** nos campos de email de Participante, Organizador e Orador (RC3), e na associação (**id_participante**, **id_evento**) de Inscricao (RC1), bem como no **id_inscricao** de Pagamento (RC2);
- **Restrições CHECK** para validar domínios, nomeadamente o tipo de evento, o estado da inscrição e do pagamento, o método de pagamento, a coerência das datas do evento e dos horários das sessões, e a positividade da capacidade e do valor do pagamento.

De entre as decisões de implementação mais relevantes destacam-se: (i) a adição do atributo `numero_edicao` à tabela `Evento`, exigido explicitamente pelo RD1; (ii) a definição de `sala` como NOT NULL em `Sessao`, necessária para garantir a aplicação da restrição RC6; e (iii) a eliminação de uma tabela intermédia `Tem` que havia sido considerada em versões anteriores, substituída pela chave estrangeira `id_evento` diretamente na tabela `Sessao`, dado que a relação entre sessão e evento é 1:N.

5.2 Criação de Utilizadores da Base de Dados

De acordo com os Requisitos de Controlo RC4 e RC5, o acesso à base de dados deve ser diferenciado consoante o perfil do utilizador. Foram criados três perfis com privilégios distintos:

Utilizador	Perfil	Privilégios
admin_eventsync	Administrador	Todos os privilégios sobre a base de dados <code>eventsync</code> (CREATE, DROP, INSERT, UPDATE, DELETE, SELECT, EXECUTE)
org_eventsync	Organizador	INSERT, UPDATE, DELETE e SELECT sobre todas as tabelas; EXECUTE sobre procedimentos e funções
readonly_eventsync	Participante / Consulta	SELECT sobre todas as tabelas e vistas

Table 23: Utilizadores da base de dados EvenSync e respetivos privilégios

A criação destes utilizadores em MySQL é realizada com os seguintes comandos:

```
CREATE USER 'admin_eventsync'@'localhost' IDENTIFIED BY 'Admin_2026!';
GRANT ALL PRIVILEGES ON eventsync.* TO 'admin_eventsync'@'localhost';

CREATE USER 'org_eventsync'@'localhost' IDENTIFIED BY 'Org_2026!';
GRANT SELECT, INSERT, UPDATE, DELETE, EXECUTE
    ON eventsync.* TO 'org_eventsync'@'localhost';

CREATE USER 'readonly_eventsync'@'localhost' IDENTIFIED BY 'Read_2026!';
GRANT SELECT ON eventsync.* TO 'readonly_eventsync'@'localhost';

FLUSH PRIVILEGES;
```

Esta separação de privilégios garante que utilizadores sem perfil de organizador não conseguem alterar dados sensíveis, assegurando os requisitos RC4 e RC5.

5.3 Povoamento da Base de Dados

A base de dados foi povoada com dados representativos do domínio do EvenSync, com o objetivo de cobrir os diversos cenários de teste necessários para validar os Requisitos de Manipulação (RM1–RM10) e as regras de integridade (RC1–RC6).

O conjunto de dados de teste inclui:

- **6 Organizadores**, incluindo docentes do Departamento de Informática e o Centro de Computação Gráfica;
- **7 Eventos** de tipos variados (conferência, workshop, corporativo, seminário), com diferentes capacidades e durações, para testar os filtros do RM1;
- **8 Sessões**, distribuídas por múltiplas salas e datas, cobrindo cenários de sobreposição controlada para validar o RC6;
- **8 Oradores** com especialidades distintas, associados a sessões através da tabela *Apresentada*;
- **10 Participantes**, provenientes de diferentes organizações (UMinho, empresas externas, participantes individuais);
- **14 Inscrições** com estados diversificados (*confirmada*, *pendente*, *cancelada*), para testar RM3, RM4 e RM10;
- **14 Pagamentos** com métodos e estados distintos (MBWay, cartão, transferência, numerário; pago, pendente, rejeitado), para testar RM5 e RM10. Todas as inscrições têm um pagamento associado, refletindo a participação total do lado do Pagamento no relacionamento R3.

A diversidade dos dados garante que todos os requisitos de manipulação possuem dados suficientes para produzir resultados não triviais nas respetivas interrogações SQL.

5.4 Cálculo do Espaço da Base de Dados (Inicial e Taxa de Crescimento Anual)

Para estimar o espaço ocupado pela base de dados EvenSync, foi efetuado um cálculo detalhado do número de bytes por linha em cada tabela, com base nos tipos de dados definidos no modelo físico. De seguida apresentam-se as regras de ocupação dos principais tipos utilizados no MySQL 8:

Tipo de dados	Espaço ocupado
INT	4 B (fixo)
DATE	3 B (fixo)
TIME	3 B (fixo)
DATETIME	8 B (fixo)
DECIMAL(8,2)	4 B (parte inteira 6 dígitos = 3B + parte decimal 2 dígitos = 1B)
VARCHAR(N)	até $N + 1$ B (pior caso; 1 B para armazenar o comprimento)
TEXT	≈ 500 B (estimativa média conservadora)

Table 24: Regras de ocupação dos tipos de dados no MySQL 8

Para o tipo `VARCHAR(N)`, é adotado o pior caso em todas as colunas ($N + 1$ bytes), garantindo que a estimativa seja conservadora e não subestime o espaço necessário.

Tabela: Participante

Atributo	Tipo	Bytes
id_participante	INT	4 B
nome	VARCHAR(100)	101 B
email	VARCHAR(100)	101 B
telemovel	VARCHAR(15)	16 B
organizacao	VARCHAR(100)	101 B
Total por linha		323 B

Table 25: Espaço por linha na tabela Participante

Tabela: Evento

Atributo	Tipo	Bytes
id_evento	INT	4 B
nome	VARCHAR(150)	151 B
tipo	VARCHAR(50)	51 B
numero_edicao	INT	4 B
data_inicio	DATE	3 B
data_fim	DATE	3 B
localizacao	VARCHAR(150)	151 B
descricao	TEXT	≈500 B
capacidade	INT	4 B
Total por linha		≈871 B

Table 26: Espaço por linha na tabela Evento

Tabela: Organizador

Atributo	Tipo	Bytes
id_organizador	INT	4 B
nome	VARCHAR(100)	101 B
email	VARCHAR(100)	101 B
telefone	VARCHAR(15)	16 B
Total por linha		222 B

Table 27: Espaço por linha na tabela Organizador

Tabela: Sessao

Atributo	Tipo	Bytes
id_sessao	INT	4 B
tema	VARCHAR(150)	151 B
data	DATE	3 B
hora_inicio	TIME	3 B
hora_fim	TIME	3 B
sala	VARCHAR(50)	51 B
id_evento	INT	4 B
Total por linha		219 B

Table 28: Espaço por linha na tabela Sessao

Tabela: Orador

Atributo	Tipo	Bytes
id_orador	INT	4 B
nome	VARCHAR(100)	101 B
email	VARCHAR(100)	101 B
especialidade	VARCHAR(100)	101 B
Total por linha		307 B

Table 29: Espaço por linha na tabela Orador

Tabela: Inscricao

Atributo	Tipo	Bytes
id_inscricao	INT	4 B
data_inscricao	DATETIME	8 B
estado	VARCHAR(15)	16 B
id_participante	INT	4 B
id_evento	INT	4 B
Total por linha		36 B

Table 30: Espaço por linha na tabela Inscricao

Tabela: Pagamento

Atributo	Tipo	Bytes
id_pagamento	INT	4 B
valor	DECIMAL(8,2)	4 B
data_pagamento	DATETIME	8 B
metodo	VARCHAR(20)	21 B
estado	VARCHAR(15)	16 B
id_inscricao	INT	4 B
Total por linha		57 B

Table 31: Espaço por linha na tabela Pagamento

Tabelas Associativas: Gere e Apresentada

As tabelas associativas contêm apenas chaves estrangeiras do tipo INT (4 B cada):

Tabela	Atributos	Bytes/linha
Gere	id_organizador (4B) + id_evento (4B)	8 B
Apresentada	id_sessao (4B) + id_orador (4B)	8 B

Table 32: Espaço por linha nas tabelas associativas

Estimativa Global do Espaço Inicial

Com base nos valores calculados acima e nas estimativas de número de registos iniciais numa instalação real do EvenSync, obtém-se a seguinte estimativa global:

Tabela	Bytes/linha	Registos iniciais	Total (KB)
Participante	323	500	≈158 KB
Evento	871	50	≈43 KB
Organizador	222	20	≈4 KB
Sessao	219	200	≈43 KB
Orador	307	100	≈30 KB
Inscricao	36	1 000	≈35 KB
Pagamento	57	900	≈50 KB
Gere	8	80	<1 KB
Apresentada	8	400	≈3 KB
Subtotal			≈367 KB
+ 30% (índices e overhead MySQL)			≈477 KB
Total estimado			≈500 KB

Table 33: Estimativa global do espaço inicial da base de dados EvenSync

A margem de 30% sobre o subtotal deve-se ao overhead interno do MySQL (cabecinhos de página InnoDB, metadados de índices, espaço livre pré-alocado por página). Esta margem é conservadora e adequada para instalações de pequena e média dimensão.

Taxa de Crescimento Anual

Estimando a atividade anual típica do sistema EvenSync, com base nos requisitos levantados, projeta-se o seguinte crescimento:

Tabela	Novos registos/ano	Bytes/linha	Crescimento (KB/ano)
Participante	200	323	≈63 KB
Evento	10	871	≈9 KB
Sessao	50	219	≈11 KB
Orador	20	307	≈6 KB
Inscricao	1 500	36	≈53 KB
Pagamento	1 500	57	≈83 KB
Gere + Apresentada	200	8	≈2 KB
Total anual			≈227 KB/ano
+ 30% overhead			≈295 KB/ano

Table 34: Taxa de crescimento anual estimada da base de dados EvenSync

Ao fim de 5 anos de operação, o volume total estimado da base de dados não deverá ultrapassar os **2 MB** ($\approx 500 \text{ KB} + 5 \times 295 \text{ KB}$), o que confirma que a base de dados do EvenSync é extremamente compacta, não requerendo estratégias de particionamento ou arquivamento de dados a curto ou médio prazo.

5.5 Definição e Caracterização de Vistas de Utilização em SQL

Foram criadas três vistas com o objetivo de simplificar o acesso a informação frequentemente consultada, proteger a estrutura interna das tabelas e facilitar a implementação dos requisitos de manipulação.

Vista v_ProgramaDetalhado (RM2). Apresenta o programa completo de cada evento, listando as sessões com os seus horários, sala e oradores associados, através de concatenação agrupada.

```
CREATE OR REPLACE VIEW v_ProgramaDetalhado AS
SELECT e.nome AS Evento, s.tema AS Sessao,
       s.data AS Dia, s.hora_inicio AS Inicio,
       s.sala AS Sala,
       GROUP_CONCAT(o.nome SEPARATOR ' | ') AS Oradores
FROM Evento e
JOIN Sessao s ON e.id_evento = s.id_evento
LEFT JOIN Apresentada apr ON s.id_sessao = apr.id_sessao
LEFT JOIN Orador o ON apr.id_orador = o.id_orador
GROUP BY s.id_sessao;
```

Vista v_DashboardFinanceiro (RM5, RM10). Agrega a informação de pagamentos por participante e evento, permitindo a consulta de receitas e pagamentos pendentes sem aceder diretamente às tabelas de inscrições e pagamentos.

```
CREATE OR REPLACE VIEW v_DashboardFinanceiro AS
SELECT p.nome AS Participante, e.nome AS Evento,
       pag.valor AS Montante, pag.estado AS Estado_Pagamento,
       pag.metodo AS Metodo
FROM Participante p
JOIN Inscricao i ON p.id_participante = i.id_participante
JOIN Evento e ON i.id_evento = e.id_evento
JOIN Pagamento pag ON i.id_inscricao = pag.id_inscricao;
```

Vista v_OcupacaoEventos (RM6). Calcula a taxa de ocupação de cada evento, com base no rácio entre inscrições confirmadas e capacidade máxima.

```
CREATE OR REPLACE VIEW v_OcupacaoEventos AS
SELECT e.nome AS Evento, e.capacidade AS Lotacao_Maxima,
       COUNT(i.id_inscricao) AS Inscritos_Confirmados,
       ROUND((COUNT(i.id_inscricao)/e.capacidade)*100, 2)
       AS Percentagem_Ocupacao
FROM Evento e
LEFT JOIN Inscricao i
      ON e.id_evento = i.id_evento AND i.estado = 'confirmada'
GROUP BY e.id_evento;
```

5.6 Tradução das Interrogações do Utilizador para SQL

Os dez Requisitos de Manipulação (RM1–RM10) foram traduzidos para SQL, recorrendo às tabelas e às vistas definidas. Apresentam-se de seguida as interrogações correspondentes.

RM1 — Listar todos os eventos, com possibilidade de filtro por tipo, data ou localização:

```
SELECT id_evento, nome, tipo, numero_edicao,
       data_inicio, data_fim, localizacao, capacidade
FROM Evento
WHERE tipo = 'Workshop'           -- filtro opcional por tipo
      AND data_inicio >= '2026-01-01' -- filtro opcional por data
ORDER BY data_inicio;
```

RM2 — Programa completo de um evento (recorre à vista):

```
SELECT * FROM v_ProgramaDetalhado
WHERE Evento = 'Workshop Flutter & Dart';
```

RM3 — Inscritos num determinado evento com estado da inscrição:

```

SELECT p.nome, p.email, i.estado, i.data_inscricao
FROM Participante p
JOIN Inscricao i ON p.id_participante = i.id_participante
JOIN Evento e    ON i.id_evento = e.id_evento
WHERE e.nome = 'Congresso Nacional de Cibersegurança'
ORDER BY i.estado, p.nome;

```

RM4 — Histórico de inscrições de um participante:

```

SELECT e.nome AS Evento, e.data_inicio, i.estado,
       i.data_inscricao
FROM Inscricao i
JOIN Evento e ON i.id_evento = e.id_evento
WHERE i.id_participante = 5
ORDER BY i.data_inscricao DESC;

```

RM5 — Total de receitas de um evento (pagamentos pagos):

```

SELECT e.nome AS Evento,
       SUM(pag.valor) AS Total_Receitas
FROM Evento e
JOIN Inscricao i    ON e.id_evento = i.id_evento
JOIN Pagamento pag ON i.id_inscricao = pag.id_inscricao
WHERE pag.estado = 'pago'
      AND e.nome = 'Workshop Flutter & Dart'
GROUP BY e.id_evento;

```

RM6 — Eventos com maior taxa de ocupação:

```

SELECT * FROM v_OcupacaoEventos
ORDER BY Percentagem_Ocupacao DESC;

```

RM7 — Oradores de um determinado evento:

```

SELECT DISTINCT o.nome, o.especialidade
FROM Orador o
JOIN Apresentada apr ON o.id_orador = apr.id_orador
JOIN Sessao s ON apr.id_sessao = s.id_sessao
JOIN Evento e ON s.id_evento = e.id_evento
WHERE e.nome = 'Congresso Nacional de Cibersegurança';

```

RM8 — Sessões agendadas numa sala ou intervalo de datas:

```

SELECT s.tema, s.data, s.hora_inicio, s.hora_fim,
       e.nome AS Evento
FROM Sessao s
JOIN Evento e ON s.id_evento = e.id_evento
WHERE s.sala = 'Auditório B1'
       OR (s.data BETWEEN '2026-06-01' AND '2026-06-30')
ORDER BY s.data, s.hora_inicio;

```

RM9 — Oradores de uma determinada sessão:

```

SELECT o.nome, o.email, o.especialidade
FROM Orador o
JOIN Apresentada apr ON o.id_orador = apr.id_orador
WHERE apr.id_sessao = 6;

```

RM10 — Pagamentos pendentes ou rejeitados, com participante e evento:

```

SELECT pag_v.Participante, pag_v.Evento,
       pag_v.Montante, pag_v.Estado_Pagamento, pag_v.Metodo
FROM v_DashboardFinanceiro pag_v
WHERE pag_v.Estado_Pagamento IN ('pendente', 'rejeitado');

```

Todas as interrogações foram testadas com o conjunto de dados de povoamento e produzem resultados corretos e não triviais.

5.7 Indexação do Sistema de Dados

Os índices têm como objetivo acelerar as consultas mais frequentes, identificadas a partir dos Requisitos de Manipulação. Para além dos índices automaticamente criados pelo MySQL sobre as chaves primárias e estrangeiras, foram definidos os seguintes índices adicionais:

Índice	Tabela	Coluna(s)	Justificação (RM)
idx_evento_tipo	Evento	tipo	RM1 — filtro por tipo
idx_evento_datas	Evento	data_inicio, data_fim	RM1, RM8 — filtro por datas
idx_sessao_sala_data	Sessao	sala, data	RM8, RC6 — consulta e controlo de s
idx_insc_estado	Inscricao	estado	RM3, RM6 — filtro por estado
idx_pag_estado	Pagamento	estado	RM5, RM10 — filtro por estado

Table 35: Índices adicionais definidos no EvenSync

A criação destes índices é realizada com os seguintes comandos:

```

CREATE INDEX idx_evento_tipo          ON Evento(tipo);
CREATE INDEX idx_evento_datas         ON Evento(data_inicio, data_fim);
CREATE INDEX idx_sessao_sala_data     ON Sessao(sala, data);
CREATE INDEX idx_insc_estado         ON Inscricao(estado);
CREATE INDEX idx_pag_estado          ON Pagamento(estado);

```

O índice composto `idx_sessao_sala_data` é particularmente relevante porque é utilizado tanto nas consultas do RM8 como na verificação de sobreposições pelo gatilho que implementa o RC6, tornando essa verificação eficiente mesmo com um grande número de sessões registradas.

5.8 Implementação de Procedimentos, Funções e Gatilhos

Foram implementados três elementos de lógica de base de dados que automatizam regras de negócio críticas: um gatilho, um procedimento armazenado e uma função.

5.8.1 Gatilho: `trg_verificar_sala_disponivel` (RC6)

Este gatilho é ativado `BEFORE INSERT` na tabela `Sessao` e verifica se existe sobreposição de horário na mesma sala e data. Caso exista, impede o registo e emite uma mensagem de erro, cumprindo o Requisito de Controlo RC6.

```

DELIMITER $$
CREATE TRIGGER trg_verificar_sala_disponivel
BEFORE INSERT ON Sessao
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1 FROM Sessao
        WHERE sala = NEW.sala
          AND data = NEW.data
          AND (NEW.hora_inicio < hora_fim
              AND NEW.hora_fim > hora_inicio)
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
            'Erro: A sala já está ocupada neste horário!';
    END IF;
END$$
DELIMITER ;

```


5.8.2 Procedimento: sp_confirmar_pagamento

Este procedimento armazenado recebe o identificador de um pagamento, atualiza o seu estado para 'pago' e, em cascata, atualiza o estado da inscrição associada para 'confirmada'. Automatiza um fluxo operacional frequente e garante a consistência entre as duas tabelas.

```
DELIMITER $$
CREATE PROCEDURE sp_confirmar_pagamento(IN p_id_pagamento INT)
BEGIN
    UPDATE Pagamento
        SET estado = 'pago', data_pagamento = NOW()
        WHERE id_pagamento = p_id_pagamento;

    UPDATE Inscricao i
    JOIN Pagamento p ON i.id_inscricao = p.id_inscricao
        SET i.estado = 'confirmada'
        WHERE p.id_pagamento = p_id_pagamento;
END$$
DELIMITER ;
```

A invocação é feita com: `CALL sp_confirmar_pagamento(3);`

5.8.3 Função: fn_total_gasto_participante

Esta função calcula o total de montantes pagos por um dado participante, somando todos os pagamentos com estado 'pago' associados às suas inscrições. Suporta diretamente o RM5 e pode ser integrada em qualquer consulta ou vista.

```
DELIMITER $$
CREATE FUNCTION fn_total_gasto_participante(
    p_id_participante INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_total DECIMAL(10,2);
    SELECT SUM(p.valor) INTO v_total
    FROM Pagamento p
    JOIN Inscricao i ON p.id_inscricao = i.id_inscricao
    WHERE i.id_participante = p_id_participante
    AND p.estado = 'pago';
    RETURN IFNULL(v_total, 0);
END$$
```

```
END$$
DELIMITER ;
```

Exemplo de utilização: `SELECT fn_total_gasto_participante(5);`

A tabela 36 resume os elementos de lógica implementados e a sua correspondência com os requisitos.

Elemento	Tipo	Requisito(s)
trg_verificar_sala_disponivel	Gatilho	RC6
sp_confirmar_pagamento	Procedimento	RC2, RD8, RD9
fn_total_gasto_participante	Função	RM5

Table 36: Elementos de lógica implementados no EvenSync

6 Conclusão e Trabalho Futuro

O presente relatório documenta o processo de concepção e implementação do sistema de base de dados relacional para a plataforma EvenSync, desenvolvido a pedido de António Carlos Silva Abelha com o objetivo de centralizar e estruturar toda a informação associada à gestão de eventos académicos e profissionais.

Ao longo das diferentes fases do projeto, a equipa percorreu o ciclo completo de desenvolvimento de um sistema de base de dados: desde o levantamento e análise de requisitos, passando pela modelação conceptual com o diagrama Entidade-Relacionamento, pela tradução para o modelo lógico relacional e respetiva normalização até à Terceira Forma Normal (3FN), até à implementação física em MySQL. Em cada etapa foram tomadas decisões técnicas devidamente justificadas, tendo sempre como referência os requisitos identificados e as boas práticas da engenharia de bases de dados.

O modelo físico resultante é composto por nove tabelas — Participante, Evento, Organizador, Sessao, Orador, Inscricao, Pagamento, Gere e Apresentada — que cobrem integralmente os dez Requisitos de Descrição identificados. A integridade dos dados é assegurada por restrições de chave primária e estrangeira, restrições de domínio (CHECK) e restrições de unicidade, complementadas por um conjunto de gatilhos, procedimentos e funções que automatizam regras de negócio críticas, como a verificação de sobreposição de sessões na mesma sala e a atualização automática do estado de inscrição após confirmação de pagamento.

A validação do modelo lógico através da álgebra relacional, utilizando a ferramenta Relax, confirmou que todos os dez Requisitos de Manipulação (RM1–RM10) são expressíveis com base nas relações e atributos definidos, o que demonstra a completude e adequação do modelo ao problema em causa. A tradução posterior para SQL, com recurso a vistas de utilização e a interrogações parametrizadas, permitiu concretizar esses requisitos num sistema funcional e testado.

Em termos globais, considera-se que os objetivos definidos no Capítulo 1 foram atingidos na sua totalidade. O sistema desenvolvido é tecnicamente

robusto, normalizado, eficiente e alinhado com as necessidades reais do cliente.

Trabalho Futuro. Apesar do sistema desenvolvido cobrir com sucesso todos os requisitos identificados, existem várias dimensões em que o EvenSync poderá ser evoluído em trabalho futuro:

- **Emissão de certificados de participação:** o modelo atual não contempla a geração ou o registo de certificados para os participantes. Uma extensão natural seria adicionar uma entidade **Certificado** associada à inscrição, com data de emissão e código de validação único;
- **Avaliação de eventos e oradores:** seria útil incorporar um mecanismo de feedback que permitisse aos participantes avaliar sessões e oradores, enriquecendo a análise pós-evento e apoiando a melhoria contínua;
- **Suporte multilíngue e internacionalização:** a base de dados encontra-se preparada para conteúdos em português, mas uma evolução para um contexto internacional exigiria a adaptação do esquema para suportar múltiplos idiomas nos campos descritivos;
- **Integração com sistemas externos:** a exposição dos dados via API REST permitiria integrar o EvenSync com plataformas de pagamento, sistemas de envio de email ou ferramentas de gestão académica já existentes nas instituições;
- **Auditoria e rastreabilidade:** a adição de tabelas de auditoria (*audit log*) que registassem todas as alterações ao estado de inscrições e pagamentos aumentaria a transparência e facilitaria a resolução de disputas.

7 Bibliografia

References

- [1] Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill.
- [2] Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
- [3] Date, C. J. (2004). *An Introduction to Database Systems* (8th ed.). Addison-Wesley.
- [4] Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>
- [5] Guter, J., & Specht, G. (2022). *RelaX — Relational Algebra Calculator*. Disponível em: <https://dbis-uibk.github.io/relax/>
- [6] Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. *Communications of the ACM*, 13(6), 377–387.

- [7] Eventbrite. (2024). *Eventbrite — Discover and Create Events Worth Experiencing*. Disponível em: <https://www.eventbrite.com/>
- [8] Meetup. (2024). *Meetup — Find events near you*. Disponível em: <https://www.meetup.com/>

A Script DDL — Criação do Esquema Físico

O script seguinte cria a base de dados `eventsync` e todas as tabelas, restrições e chaves estrangeiras do modelo físico do EvenSync (MySQL 8.x).

```
-- =====
-- EvenSync - Script de Criação do Esquema (Modelo Físico)
-- Base de Dados - LCC 2025/2026
-- SGBD: MySQL 8.x
--
-- CORREÇÕES APLICADAS (v2):
-- [C1] Evento      - adicionado numero_edicao (RD1 exige-o explicitamente)
-- [C2] Participante - telemovel passou a NOT NULL (RD7: campo obrigatório)
-- [C3] Sessao      - adicionado id_evento FK (1:N em vez de N:M)
--                  - sala passou a NOT NULL (RC6: controlo de sobreposição)
-- [C4] Tabela Tem  - REMOVIDA (era N:M sem base nos requisitos;
--                  - substituída pela FK id_evento em Sessao)
-- [OK] Pagamento  - uq_pag_inscricao (UNIQUE) já garantia o 1:1; mantido
--
-- CORREÇÕES APLICADAS (v3):
-- [C5] Evento.tipo - CONSTRAINT CHECK adicionada para restringir os
--                  valores aceites a: 'conferencia', 'workshop',
--                  'corporativo', 'seminario'
-- =====

DROP DATABASE IF EXISTS eventsync;
CREATE DATABASE eventsync
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE eventsync;

-- -----
-- 1. Participante
-- RD7: nome, email, telefone e organização
-- [C2] telemovel: NULL → NOT NULL
-- -----
CREATE TABLE Participante (
    id_participante INT NOT NULL AUTO_INCREMENT,
    nome            VARCHAR(100) NOT NULL,
    email           VARCHAR(100) NOT NULL,
    telemovel       VARCHAR(15)  NOT NULL,          -- [C2] era NULL
    organizacao     VARCHAR(100)  NULL,

    CONSTRAINT pk_participante PRIMARY KEY (id_participante),
    CONSTRAINT uq_participante_email UNIQUE (email)
);

-- -----
-- 2. Evento
```

```

-- RD1: id, nome, tipo, numero_edicao, data_inicio, data_fim,
--      localizacao, capacidade, descricao
-- [C1] numero_edicao: campo novo (estava omitido)
-- [C5] tipo: restrito a conferencia | workshop | corporativo | seminario
-----
CREATE TABLE Evento (
    id_evento      INT          NOT NULL AUTO_INCREMENT,
    nome           VARCHAR(150) NOT NULL,
    tipo           VARCHAR(50)  NOT NULL,
    numero_edicao   INT          NOT NULL,           -- [C1] novo campo
    data_inicio    DATE         NOT NULL,
    data_fim       DATE         NOT NULL,
    localizacao    VARCHAR(150) NOT NULL,
    descricao      TEXT         NULL,
    capacidade     INT          NOT NULL,

    CONSTRAINT pk_evento          PRIMARY KEY (id_evento),
    CONSTRAINT ck_evento_tipo     CHECK (tipo IN ('conferencia','workshop','corporativo','seminario')),
    CONSTRAINT ck_evento_datas    CHECK (data_inicio <= data_fim),
    CONSTRAINT ck_evento_capacidade CHECK (capacidade > 0),
    CONSTRAINT ck_evento_edicao     CHECK (numero_edicao > 0)
);

-----
-- 3. Organizador
-- RD2: id, nome, email, telefone
-----
CREATE TABLE Organizador (
    id_organizador INT          NOT NULL AUTO_INCREMENT,
    nome           VARCHAR(100) NOT NULL,
    email          VARCHAR(100) NOT NULL,
    telefone       VARCHAR(15)  NOT NULL,

    CONSTRAINT pk_organizador     PRIMARY KEY (id_organizador),
    CONSTRAINT uq_organizador_email UNIQUE      (email)
);

-----
-- 4. Sessao
-- RD4: id, titulo, sala, data, hora_inicio, hora_fim, descricao
-- RC6: não pode haver sobreposição de sessões na mesma sala
-- [C3a] id_evento INT NOT NULL FK → Evento (1:N - RD4 "associadas a UM evento")
-- [C3b] sala: NULL → NOT NULL (RC6 pressupõe sala sempre preenchida)
-- [C4] tabela Tem removida; relação gerida por este FK
-----
CREATE TABLE Sessao (
    id_sessao      INT          NOT NULL AUTO_INCREMENT,
    tema           VARCHAR(150) NOT NULL,
    data           DATE         NOT NULL,
    hora_inicio    TIME         NOT NULL,

```

```

hora_fim    TIME          NOT NULL,
sala        VARCHAR(50)   NOT NULL,          -- [C3b] era NULL
id_evento   INT           NOT NULL,          -- [C3a] novo FK

CONSTRAINT pk_sessao      PRIMARY KEY (id_sessao),
CONSTRAINT fk_sessao_evento FOREIGN KEY (id_evento)
    REFERENCES Evento(id_evento)
    ON UPDATE CASCADE ON DELETE CASCADE,
CONSTRAINT ck_sessao_horas CHECK (hora_inicio < hora_fim)
);

-----
-- 5. Orador
--   RD5: id, nome, email, biografia, especialidade
-----
CREATE TABLE Orador (
    id_orador    INT          NOT NULL AUTO_INCREMENT,
    nome         VARCHAR(100) NOT NULL,
    email        VARCHAR(100) NOT NULL,
    especialidade VARCHAR(100) NOT NULL,

    CONSTRAINT pk_orador      PRIMARY KEY (id_orador),
    CONSTRAINT uq_orador_email UNIQUE      (email)
);

-----
-- 6. Inscricao (R1: Participante 1-N; R2: Evento 1-N)
--   RD8: id, data_inscricao, estado
--   RC1: um participante não pode ter mais de uma inscrição activa
--         no mesmo evento → UNIQUE(id_participante, id_evento)
-----
CREATE TABLE Inscricao (
    id_inscricao    INT          NOT NULL AUTO_INCREMENT,
    data_inscricao   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    estado           VARCHAR(15)  NOT NULL DEFAULT 'pendente',
    id_participante  INT          NOT NULL,
    id_evento        INT          NOT NULL,

    CONSTRAINT pk_inscricao    PRIMARY KEY (id_inscricao),
    CONSTRAINT fk_insc_part    FOREIGN KEY (id_participante)
        REFERENCES Participante(id_participante)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_insc_evento  FOREIGN KEY (id_evento)
        REFERENCES Evento(id_evento)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT uq_insc_part_evento UNIQUE (id_participante, id_evento), -- RC1
    CONSTRAINT ck_insc_estado  CHECK (estado IN ('pendente','confirmada','cancelada'))
);

-----

```

```

-- 7. Pagamento (R3: Inscricao 1-1)
-- RD9: id, valor, data_pagamento, metodo, estado
-- RC2: cada inscrição tem no máximo um pagamento
-- [OK] uq_pag_inscricao (UNIQUE) já existia e garante o 1:1
-----
CREATE TABLE Pagamento (
    id_pagamento INT NOT NULL AUTO_INCREMENT,
    valor DECIMAL(8,2) NOT NULL,
    data_pagamento DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    metodo VARCHAR(20) NOT NULL,
    estado VARCHAR(15) NOT NULL DEFAULT 'pendente',
    id_inscricao INT NOT NULL,

    CONSTRAINT pk_pagamento PRIMARY KEY (id_pagamento),
    CONSTRAINT fk_pag_insc FOREIGN KEY (id_inscricao)
        REFERENCES Inscricao(id_inscricao)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT uq_pag_inscricao UNIQUE (id_inscricao), -- RC2 / 1:1
    CONSTRAINT ck_pag_metodo CHECK (metodo IN ('cartao','transferencia','MBWay','numerario')),
    CONSTRAINT ck_pag_estado CHECK (estado IN ('pendente','pago','rejeitado')),
    CONSTRAINT ck_pag_valor CHECK (valor > 0)
);

-----
-- 8. Gere (R4: Organizador N-M Evento)
-- RD3: um organizador pode gerir múltiplos eventos e vice-versa
-----
CREATE TABLE Gere (
    id_organizador INT NOT NULL,
    id_evento INT NOT NULL,

    CONSTRAINT pk_gere PRIMARY KEY (id_organizador, id_evento),
    CONSTRAINT fk_gere_org FOREIGN KEY (id_organizador)
        REFERENCES Organizador(id_organizador)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_gere_evento FOREIGN KEY (id_evento)
        REFERENCES Evento(id_evento)
        ON UPDATE CASCADE ON DELETE CASCADE
);

-----
-- 9. Apresentada (R6: Sessao N-M Orador)
-- RD6: uma sessão pode ter vários oradores; um orador pode
-- apresentar em várias sessões
-----
CREATE TABLE Apresentada (
    id_sessao INT NOT NULL,
    id_orador INT NOT NULL,

    CONSTRAINT pk_apresentada PRIMARY KEY (id_sessao, id_orador),

```



```

        CONSTRAINT fk_apr_sessao FOREIGN KEY (id_sessao)
        REFERENCES Sessao(id_sessao)
        ON UPDATE CASCADE ON DELETE CASCADE,
        CONSTRAINT fk_apr_orador FOREIGN KEY (id_orador)
        REFERENCES Orador(id_orador)
        ON UPDATE CASCADE ON DELETE CASCADE
    );

-- =====
-- Verificação: listar tabelas criadas (deve mostrar 9 tabelas)
-- Participante, Evento, Organizador, Sessao, Orador,
-- Inscricao, Pagamento, Gere, Apresentada
-- =====
SHOW TABLES;

```

B Script de Povoamento

O script seguinte insere o conjunto de dados de teste utilizado para validar os Requisitos de Manipulação (RM1–RM10) e as regras de integridade (RC1–RC6).

```
-- =====
-- EvenSync - Povoamento da Base de Dados
-- Base de Dados - LCC 2025/2026
-- 6 Organizadores | 7 Eventos | 8 Sessões | 8 Oradores
-- 10 Participantes | 14 Inscrições | 14 Pagamentos
-- =====

USE eventsync;

-- -----
-- 1. ORGANIZADORES (IDs 1-6)
-- -----
INSERT INTO Organizador (nome, email, telefone) VALUES
('José Bumblebee', 'jose.bumblebee@eventsync.pt', '253000001'),
('André Marques', 'a111634@alunos.uminho.pt', '253604401'),
('Bruno Freitas', 'a110016@alunos.uminho.pt', '253604402'),
('João Ribeiro', 'a111041@alunos.uminho.pt', '253604403'),
('Tiago Santos', 'a110313@alunos.uminho.pt', '253604404'),
('Nelson Antunes', 'a110360@alunos.uminho.pt', '253604405');

-- -----
-- 2. EVENTOS (IDs 1-7)
-- Cobre os 4 tipos permitidos: conferencia, workshop,
-- corporativo, seminario
-- -----
INSERT INTO Evento (nome, tipo, numero_edicao, data_inicio, data_fim, localizacao, capacidade, descricao) VALUES
('Workshop Flutter & Dart', 'workshop', 2, '2026-05-25', '2026-05-25', 'DI-A1', 10, 'Workshop de Flutter & Dart'),
('Congresso Nacional de Cibersegurança', 'conferencia', 5, '2026-06-10', '2026-06-12', 'Auditório B', 50, 'Congresso Nacional de Cibersegurança'),
('Hackathon UMinho 2026', 'workshop', 1, '2026-09-01', '2026-09-03', 'CP1', 30, 'Hackathon UMinho 2026'),
('Sessão Solene de Abertura LCC', 'corporativo', 32, '2026-09-15', '2026-09-15', 'Auditório B', 100, 'Sessão Solene de Abertura LCC'),
('Seminário de Bases de Dados', 'seminario', 3, '2026-10-05', '2026-10-05', 'DI-B1', 20, 'Seminário de Bases de Dados'),
('Encontro de Inovação Empresarial', 'corporativo', 7, '2026-10-20', '2026-10-21', 'Auditório B', 50, 'Encontro de Inovação Empresarial'),
('Conferência de Inteligência Artificial', 'conferencia', 2, '2026-11-20', '2026-11-22', 'Auditório B', 50, 'Conferência de Inteligência Artificial');

-- -----
-- 3. GERE - Organizador gere Evento (N:M)
-- -----
INSERT INTO Gere (id_organizador, id_evento) VALUES
(1, 1), (1, 2), -- José gere Workshop e Congresso
(2, 3), -- André gere Hackathon
(3, 4), (6, 4), -- Bruno e Nelson gerem Sessão Solene
(4, 5), (4, 6), -- João gere Seminário e Encontro
(5, 7); -- Tiago gere Conferência de IA
```

-- 4. ORADORES (IDs 1-8)

```
-----
INSERT INTO Orador (nome, email, especialidade) VALUES
('Prof. Alberto Silva', 'alberto@di.uminho.pt', 'Engenharia de Software'),
('Eng. Maria Fontes', 'mfontes@checkpoint.com', 'Network Security'),
('Dr. Ricardo Jorge', 'rjorge@google.com', 'Cloud Computing'),
('Sara Antunes', 'sara@flutter.dev', 'Mobile Dev'),
('Nelson Antunes', 'nantunes@di.uminho.pt', 'Distributed Databases'),
('Prof. Paulo Martins', 'pmartins@di.uminho.pt', 'Inteligência Artificial'),
('Dra. Sofia Costa', 'scosta@inovacao.pt', 'Inovação Empresarial'),
('Eng. Rui Pereira', 'rpereira@checkpoint.com', 'Cibersegurança');
```

-- 5. SESSÕES (IDs 1-8)

```
-----
INSERT INTO Sessao (tema, data, hora_inicio, hora_fim, sala, id_evento) VALUES
('Introdução ao Flutter', '2026-05-25', '09:00:00', '11:00:00', 'DI-A1', 1),
('Widgets Avançados', '2026-05-25', '11:30:00', '13:00:00', 'DI-A1', 1),
('Painel: IA e Cibersegurança', '2026-06-10', '14:30:00', '16:00:00', 'Auditório B1', 2),
('Criptografia Quântica', '2026-06-11', '10:00:00', '12:00:00', 'Auditório B1', 2),
('Final Pitch & Demo Day', '2026-09-03', '15:00:00', '18:00:00', 'CP1', 3),
('Abertura Oficial', '2026-09-15', '09:00:00', '10:00:00', 'Auditório B2', 4),
('Fundamentos de BD Relacionais', '2026-10-05', '10:00:00', '12:00:00', 'DI-B1', 5),
('Tendências em Machine Learning', '2026-11-20', '14:00:00', '16:00:00', 'Auditório B1', 7);
```

-- 6. APRESENTADA - Orador apresenta Sessão (N:M)

```
-----
INSERT INTO Apresentada (id_sessao, id_orador) VALUES
(1, 4), -- Sara na sessão de Flutter Intro
(2, 4), -- Sara em Widgets Avançados
(3, 1), (3, 2), -- Alberto e Maria no painel de IA
(4, 8), -- Rui na Criptografia Quântica
(5, 5), -- Nelson no Hackathon
(7, 5), -- Nelson no Seminário de BD
(8, 6); -- Paulo em Machine Learning
```

-- 7. PARTICIPANTES (IDs 1-10)

```
-----
INSERT INTO Participante (nome, email, telemovel, organizacao) VALUES
('Tiago Santos', 'tiago.a110313@alunos.uminho.pt', '910000001', 'UMinho'),
('Bruno Freitas', 'bruno.a110016@alunos.uminho.pt', '910000002', 'UMinho'),
('João Silva', 'jsilva@empresa.pt', '920000001', 'TechCorp'),
('Maria Oliveira', 'moliveira@gmail.com', '930000001', NULL),
('Ricardo Costa', 'rcosta@hotmail.com', '960000001', 'IT Solutions'),
('Sílvia Mendes', 'smendes@alunos.uminho.pt', '910000003', 'UMinho'),
('Nuno Rocha', 'nuno.rocha@ieee.org', '910000004', 'IEEE Student Branch'),
('Beatriz Dias', 'beatriz@gmail.com', '920000002', NULL),
('Fernando Pessoa', 'fpessoa@literatura.pt', '910000005', 'Casa Fernando Pessoa');
```

```

('Marta Ferreira', 'marta.f@uminho.pt', '253000010', 'UMinho');

-----
-- 8. INSCRIÇÕES (IDs 1-14)
-- Estados variados para testar RM3, RM4, RM10
-- UNIQUE(id_participante, id_evento) garante RC1
-----
INSERT INTO Inscricao (id_participante, id_evento, estado) VALUES
(1, 1, 'confirmada'), (2, 1, 'confirmada'), (3, 1, 'pendente'), -- Workshop Flutter
(1, 2, 'confirmada'), (4, 2, 'confirmada'), (5, 2, 'cancelada'), -- Congresso
(6, 2, 'confirmada'), (7, 2, 'confirmada'), (8, 2, 'confirmada'), -- Congresso (cont.)
(1, 3, 'confirmada'), (9, 3, 'confirmada'), (10, 3, 'confirmada'), -- Hackathon
(1, 4, 'confirmada'), (10, 4, 'confirmada'); -- Sessão Solene

-----
-- 9. PAGAMENTOS (IDs 1-14)
-- Métodos e estados distintos para testar RM5, RM10
-- uq_pag_inscricao garante RC2 (1 pagamento por inscrição)
-- Todas as inscrições têm pagamento (R3 P:T - Inscrição parcial, Pagamento total)
-----
INSERT INTO Pagamento (valor, metodo, estado, id_inscricao) VALUES
(10.00, 'MBWay', 'pago', 1), -- Tiago pagou Workshop
(10.00, 'cartao', 'pago', 2), -- Bruno pagou Workshop
(10.00, 'transferencia', 'pendente', 3), -- João Silva pagamento pendente (inscrição pendente)
(25.00, 'transferencia', 'pago', 4), -- Tiago pagou Congresso
(25.00, 'MBWay', 'pendente', 5), -- Maria pagamento pendente
(25.00, 'cartao', 'pago', 6), -- Silvia pagou Congresso
(25.00, 'cartao', 'pago', 7), -- Nuno pagou Congresso
(25.00, 'transferencia', 'pago', 8), -- Beatriz pagou Congresso
(25.00, 'MBWay', 'rejeitado', 9), -- Beatriz 2.ª pagamento rejeitado
(5.00, 'numerario', 'pago', 10), -- Tiago pagou Hackathon
(5.00, 'MBWay', 'pago', 11), -- Fernando Pessoa pagou Hackathon
(5.00, 'cartao', 'pago', 12), -- Marta pagou Hackathon
(5.00, 'numerario', 'pago', 13), -- Tiago pagou Sessão Solene
(5.00, 'MBWay', 'pago', 14); -- Marta pagou Sessão Solene

```

C Vistas, Gatilho, Procedimento, Função e Interrogações SQL

O script seguinte contém as três vistas de utilização, o gatilho de verificação de sala, o procedimento de confirmação de pagamento, a função de cálculo de total gasto e a tradução das interrogações RM1–RM10 para SQL.

```
CREATE OR REPLACE VIEW v_ProgramaDetalhado AS
SELECT
    e.nome AS Evento,
    s.tema AS Sessao,
    s.data AS Dia,
    s.hora_inicio AS Inicio,
    s.sala AS Sala,
    GROUP_CONCAT(o.nome SEPARATOR ' | ') AS Oradores
FROM Evento e
JOIN Sessao s ON e.id_evento = s.id_evento
LEFT JOIN Apresentada apr ON s.id_sessao = apr.id_sessao
LEFT JOIN Orador o ON apr.id_orador = o.id_orador
GROUP BY s.id_sessao;

CREATE OR REPLACE VIEW v_DashboardFinanceiro AS
SELECT
    p.nome AS Participante,
    e.nome AS Evento,
    pag.valor AS Montante,
    pag.estado AS Estado_Pagamento,
    pag.metodo AS Metodo
FROM Participante p
JOIN Inscricao i ON p.id_participante = i.id_participante
JOIN Evento e ON i.id_evento = e.id_evento
JOIN Pagamento pag ON i.id_inscricao = pag.id_inscricao;

CREATE OR REPLACE VIEW v_OcupacaoEventos AS
SELECT
    e.nome AS Evento,
    e.capacidade AS Lotação_Máxima,
    COUNT(i.id_inscricao) AS Inscritos_Confirmados,
    ROUND((COUNT(i.id_inscricao) / e.capacidade) * 100, 2) AS Percentagem_Ocupacao
FROM Evento e
LEFT JOIN Inscricao i ON e.id_evento = i.id_evento AND i.estado = 'confirmada'
GROUP BY e.id_evento;

DELIMITER $$

CREATE TRIGGER trg_verificar_sala_disponivel
BEFORE INSERT ON Sessao
FOR EACH ROW
BEGIN
```

```

        IF EXISTS (
            SELECT 1 FROM Sessao
            WHERE sala = NEW.sala
                AND data = NEW.data
                AND (
                    (NEW.hora_inicio < hora_fim AND NEW.hora_fim > hora_inicio)
                )
        ) THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Erro: A sala já está ocupada neste horário!';
        END IF;
    END$$

DELIMITER ;

DELIMITER $$

CREATE PROCEDURE sp_confirmar_pagamento(IN p_id_pagamento INT)
BEGIN
    -- 1. Atualiza o pagamento
    UPDATE Pagamento SET estado = 'pago', data_pagamento = NOW()
    WHERE id_pagamento = p_id_pagamento;

    -- 2. Atualiza a inscrição associada
    UPDATE Inscricao i
    JOIN Pagamento p ON i.id_inscricao = p.id_inscricao
    SET i.estado = 'confirmada'
    WHERE p.id_pagamento = p_id_pagamento;
END$$

DELIMITER ;

DELIMITER $$

CREATE FUNCTION fn_total_gasto_participante(p_id_participante INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_total DECIMAL(10,2);
    SELECT SUM(p.valor) INTO v_total
    FROM Pagamento p
    JOIN Inscricao i ON p.id_inscricao = i.id_inscricao
    WHERE i.id_participante = p_id_participante AND p.estado = 'pago';
    RETURN IFNULL(v_total, 0);
END$$

DELIMITER ;

-- =====
-- TRADUÇÃO DAS INTERROGAÇÕES DO UTILIZADOR PARA SQL (RM1-RM10)

```

```

-- =====

-- RM1: Listar todos os eventos, com filtro por tipo, data ou localização
SELECT id_evento, nome, tipo, numero_edicao,
       data_inicio, data_fim, localizacao, capacidade
FROM Evento
WHERE tipo = 'Workshop'           -- filtro opcional por tipo
      AND data_inicio >= '2026-01-01' -- filtro opcional por data
ORDER BY data_inicio;

-- RM2: Programa completo de um evento (recorre à vista)
SELECT * FROM v_ProgramaDetalhado
WHERE Evento = 'Workshop Flutter & Dart';

-- RM3: Inscritos num determinado evento com estado da inscrição
SELECT p.nome, p.email, i.estado, i.data_inscricao
FROM Participante p
JOIN Inscricao i ON p.id_participante = i.id_participante
JOIN Evento e   ON i.id_evento = e.id_evento
WHERE e.nome = 'Congresso Nacional de Cibersegurança'
ORDER BY i.estado, p.nome;

-- RM4: Histórico de inscrições de um participante
SELECT e.nome AS Evento, e.data_inicio, i.estado,
       i.data_inscricao
FROM Inscricao i
JOIN Evento e ON i.id_evento = e.id_evento
WHERE i.id_participante = 5
ORDER BY i.data_inscricao DESC;

-- RM5: Total de receitas de um evento (pagamentos com estado 'pago')
SELECT e.nome AS Evento,
       SUM(pag.valor) AS Total_Receitas
FROM Evento e
JOIN Inscricao i ON e.id_evento = i.id_evento
JOIN Pagamento pag ON i.id_inscricao = pag.id_inscricao
WHERE pag.estado = 'pago'
      AND e.nome = 'Workshop Flutter & Dart'
GROUP BY e.id_evento;

-- RM6: Eventos com maior taxa de ocupação (recorre à vista)
SELECT * FROM v_OcupacaoEventos
ORDER BY Percentagem_Ocupacao DESC;

-- RM7: Oradores de um determinado evento
SELECT DISTINCT o.nome, o.especialidade
FROM Orador o
JOIN Apresentada apr ON o.id_orador = apr.id_orador
JOIN Sessao s ON apr.id_sessao = s.id_sessao
JOIN Evento e ON s.id_evento = e.id_evento

```

```

WHERE e.nome = 'Congresso Nacional de Cibersegurança';

-- RM8: Sessões agendadas numa sala ou intervalo de datas
SELECT s.tema, s.data, s.hora_inicio, s.hora_fim,
       e.nome AS Evento
FROM Sessao s
JOIN Evento e ON s.id_evento = e.id_evento
WHERE s.sala = 'Auditório B1'
       OR (s.data BETWEEN '2026-06-01' AND '2026-06-30')
ORDER BY s.data, s.hora_inicio;

-- RM9: Oradores de uma determinada sessão
SELECT o.nome, o.email, o.especialidade
FROM Orador o
JOIN Apresentada apr ON o.id_orador = apr.id_orador
WHERE apr.id_sessao = 6;

-- RM10: Pagamentos pendentes ou rejeitados, com participante e evento
SELECT pag_v.Participante, pag_v.Evento,
       pag_v.Montante, pag_v.Estado_Pagamento, pag_v.Metodo
FROM v_DashboardFinanceiro pag_v
WHERE pag_v.Estado_Pagamento IN ('pendente', 'rejeitado');

```


D Diagrama Entidade-Relacionamento (tamanho completo)

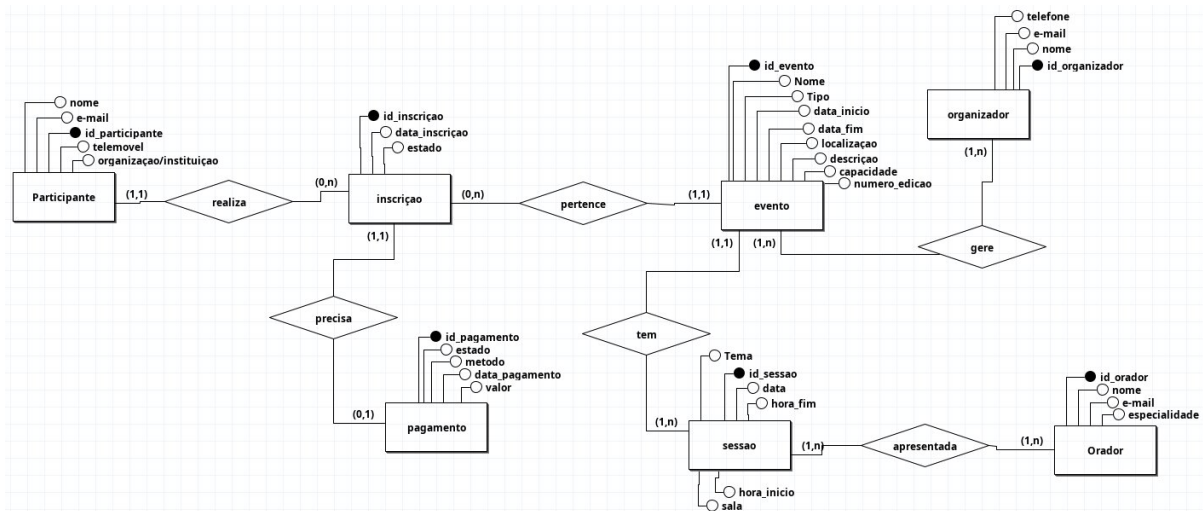


Figure 10: Diagrama ER completo do sistema EvenSync (notação de Chen)

E Modelo Lógico (tamanho completo)

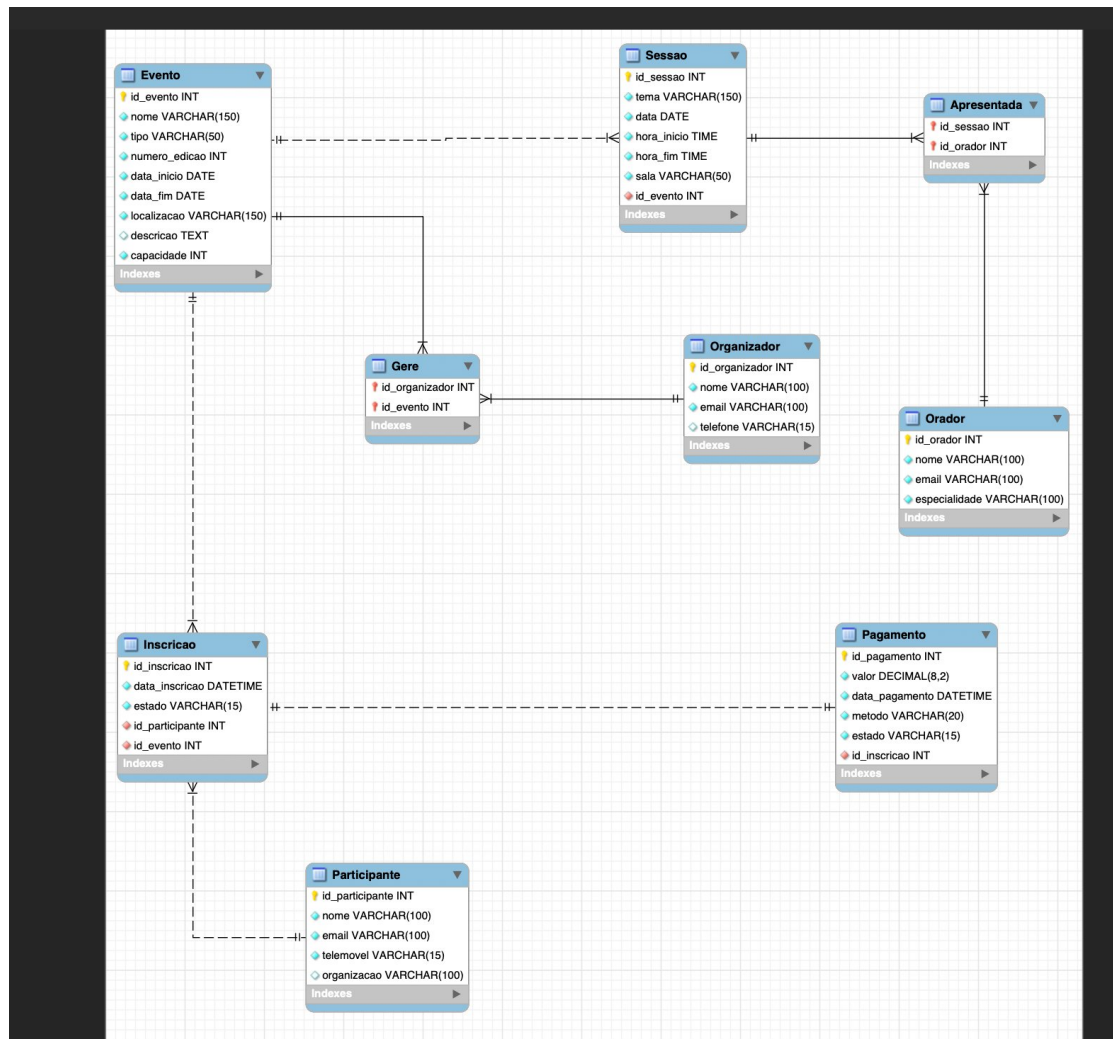


Figure 11: Modelo lógico do sistema EvenSync (gerado com MySQL Workbench)